

# Programmeren in C

Practicumopdrachten bij het vak CPROUC



Opleiding Elektrotechniek

Versie 0.96  
30 september 2026

# Inhoudsopgave

|                                                                      |           |
|----------------------------------------------------------------------|-----------|
| <b>Aanbevolen werkwijze</b>                                          | <b>5</b>  |
| <b>1 Inleiding</b>                                                   | <b>6</b>  |
| 1.1 De gebruikte C-compiler                                          | 6         |
| 1.2 Installeren Visual Studio Community en invoeren eerste programma | 6         |
| 1.3 Programma wijzigen                                               | 6         |
| 1.4 Vertalen van het gewijzigde programma                            | 6         |
| 1.5 Programma met invoer                                             | 7         |
| <b>2 Toekennings- en herhalingsopdrachten (for, while)</b>           | <b>8</b>  |
| 2.1 Inleiding                                                        | 8         |
| 2.2 Kwadraten en derde machten                                       | 9         |
| 2.3 Tafel van 12                                                     | 9         |
| 2.4 Achterstevoren                                                   | 9         |
| 2.5 Deelbaar door 9                                                  | 9         |
| 2.6 Harmonische reeks                                                | 9         |
| 2.7 Driehoek van Floyd                                               | 9         |
| 2.8 Inwendig en uitwendig vermogen                                   | 9         |
| 2.9 Parallelweerstand                                                | 10        |
| 2.10 Bazel-probleem                                                  | 10        |
| 2.11 De spaarrekening                                                | 10        |
| 2.12 De transformator                                                | 11        |
| 2.13 Faculteit                                                       | 12        |
| 2.14 Afgelegde weg                                                   | 12        |
| 2.15 Benadering sinus                                                | 13        |
| 2.16 Kerstboom                                                       | 13        |
| 2.17 Maandkalender                                                   | 13        |
| 2.18 Wind-chill temperatuur                                          | 14        |
| 2.19 Benadering e                                                    | 14        |
| 2.20 Opladen van een condensator                                     | 15        |
| 2.21 Stroom door een diode                                           | 16        |
| 2.22 Booglengte van één periode van een sinus                        | 16        |
| 2.23 Oppervlakte bepalen                                             | 17        |
| <b>3 Beslissingsopdrachten (if, if else, switch)</b>                 | <b>18</b> |
| 3.1 Inleiding                                                        | 18        |
| 3.2 Grootste getal                                                   | 19        |
| 3.3 Een na grootste getal                                            | 19        |
| 3.4 Kleinste en één na kleinste getal                                | 19        |
| 3.5 Naam van de maand afdrukken                                      | 19        |
| 3.6 Cijfer als woord afdrukken                                       | 19        |
| 3.7 Verandering van ingevoerde getallen                              | 19        |
| 3.8 Even en oneven                                                   | 19        |
| 3.9 Voldoende en onvoldoende                                         | 19        |
| 3.10 Aantal ingevoerde cijfers afdrukken                             | 20        |
| 3.11 PIN-code raden                                                  | 20        |
| 3.12 Overwerk                                                        | 20        |
| 3.13 ABC-formule                                                     | 20        |
| 3.14 Simpele rekenmachine                                            | 20        |
| 3.15 Afdrukken binair getal                                          | 21        |
| 3.16 Windsnelheden                                                   | 21        |
| 3.17 Geschreven letters                                              | 21        |
| 3.18 Stijgen                                                         | 22        |
| 3.19 Tekens tellen                                                   | 22        |

|          |                                                         |           |
|----------|---------------------------------------------------------|-----------|
| 3.20     | Dag van de week                                         | 22        |
| 3.21     | Bepalen schrikkeljaar                                   | 22        |
| 3.22     | Extra dagen                                             | 23        |
| 3.23     | Stelsel van twee lineaire vergelijkingen                | 23        |
| <b>4</b> | <b>Funcities</b>                                        | <b>24</b> |
| 4.1      | Inleiding                                               | 24        |
| 4.2      | Vermenigvuldigen                                        | 25        |
| 4.3      | Vermenigvuldigen en delen                               | 25        |
| 4.4      | Catalan-getal (iteratief)                               | 25        |
| 4.5      | Catalan-getal (recursief)                               | 25        |
| 4.6      | Vermoeden van Collatz                                   | 26        |
| 4.7      | Digitale techniek                                       | 26        |
| 4.8      | Maximale                                                | 27        |
| 4.9      | Gemiddelde                                              | 27        |
| 4.10     | Kleinste gemene veelvoud                                | 28        |
| 4.11     | Wortel schieten                                         | 28        |
| 4.12     | Som van de cijfers en modulo 9                          | 29        |
| 4.13     | Isolated prime                                          | 29        |
| 4.14     | De Ackermann-functie                                    | 29        |
| 4.15     | The sinc-functie                                        | 30        |
| 4.16     | Aantal delers van een geheel getal                      | 30        |
| 4.17     | Nulpunten van een functie bepalen                       | 30        |
| <b>5</b> | <b>Arrays</b>                                           | <b>31</b> |
| 5.1      | Inleiding                                               | 31        |
| 5.2      | Omkeren                                                 | 32        |
| 5.3      | Voortschrijden                                          | 32        |
| 5.4      | Vergelijken                                             | 32        |
| 5.5      | Histogram                                               | 32        |
| 5.6      | Sorteren met "selection sort"                           | 32        |
| 5.7      | Palindroom                                              | 33        |
| 5.8      | Sorteren met "bubble sort"                              | 33        |
| 5.9      | Zoeken                                                  | 33        |
| 5.10     | Sinustabel                                              | 33        |
| 5.11     | Cesuur aanpassen                                        | 34        |
| 5.12     | Lineaire regressie                                      | 34        |
| 5.13     | Controle identificatiecode van Nederland eurobankbiljet | 34        |
| 5.14     | Doorsnede                                               | 35        |
| 5.15     | Uniek                                                   | 35        |
| 5.16     | Run Length encoding                                     | 36        |
| 5.17     | Rolling dice                                            | 36        |
| 5.18     | Overtollige spaties en tabs verwijderen                 | 36        |
| 5.19     | Waar of niet waar                                       | 37        |
| 5.20     | Queue                                                   | 37        |
| <b>6</b> | <b>Structures</b>                                       | <b>38</b> |
| 6.1      | Inleiding                                               | 38        |
| 6.2      | Afstand tussen twee punten                              | 39        |
| 6.3      | Omtrek van een polygon                                  | 39        |
| 6.4      | Tijd naar seconden                                      | 39        |
| 6.5      | Seconden naar tijd                                      | 40        |
| 6.6      | Tijdsverschil                                           | 40        |
| 6.7      | Studenten                                               | 40        |
| 6.8      | Studenten again                                         | 40        |

|          |                                                |           |
|----------|------------------------------------------------|-----------|
| <b>7</b> | <b>Pointers</b>                                | <b>41</b> |
| 7.1      | Introductie                                    | 41        |
| 7.2      | Array afdrukken                                | 42        |
| 7.3      | Array afdrukken, reprise                       | 42        |
| 7.4      | String achterstevoren afdrukken                | 42        |
| 7.5      | Woorden tellen                                 | 42        |
| 7.6      | Array van integers omkeren                     | 42        |
| 7.7      | Zoek het grootste getal                        | 42        |
| 7.8      | Op volgorde                                    | 42        |
| 7.9      | Zoek laatste getal                             | 43        |
| 7.10     | Vervangen van karakter                         | 43        |
| 7.11     | Vervangen van woord                            | 43        |
| 7.12     | Dynamische array                               | 43        |
| 7.13     | Dynamische array omkeren                       | 43        |
| 7.14     | Dynamisch inlezen van een string               | 43        |
| 7.15     | Dynamisch inlezen van een $n \times n$ -matrix | 43        |
| 7.16     | Onbeperkt                                      | 44        |
| <b>8</b> | <b>Bestanden</b>                               | <b>45</b> |
| 8.1      | Inleiding                                      | 45        |
| 8.2      | More                                           | 46        |
| 8.3      | Cat                                            | 46        |
| 8.4      | Zinslengte sorteren                            | 46        |
| 8.5      | Op alfabet sorteren                            | 46        |
| 8.6      | Regelnummering                                 | 46        |
| 8.7      | Getallen sorteren                              | 46        |
| 8.8      | Uitgeschreven nummers sorteren                 | 46        |
| 8.9      | C keywords                                     | 47        |
| 8.10     | Commentaar verwijderen                         | 47        |
| 8.11     | Letters tellen                                 | 47        |
| 8.12     | Woorden tellen, en meer                        | 47        |
| 8.13     | Omkeren                                        | 47        |
| 8.14     | Verwijderen                                    | 47        |
| 8.15     | XeT                                            | 48        |
| 8.16     | *) Argumenten op de command-line               | 48        |
| <b>A</b> | <b>Installatie Visual Studio Community</b>     | <b>50</b> |
| <b>B</b> | <b>Installatie Code::Blocks</b>                | <b>55</b> |

## Aanbevolen werkwijze

Volg onderstaande werkwijze als je geïnteresseerd bent in aanzienlijk efficiënter en sneller tot een voldoende eindresultaat te komen. Het is natuurlijk aan jou om deze aanbevolen werkwijze te volgen:

### 1. Lees goed de opdracht

De meest gemaakte fout is dat de opdracht niet goed wordt gelezen. Stel jezelf na het lezen van de opdracht de vraag wat het programma precies moet kunnen. Wat moet je opleveren?

### 2. Teken je programma uit met pen en papier

Een andere veel gemaakte fout is dat de student ondoordacht (lees: zonder na te denken) begint met programmeren. STOP! Als je de opdracht hebt begrepen, ga je niet programmeren, maar pak je pen en papier. Als pen en papier voor je ligt, ga je uitschrijven, uitdenken en uittekenen hoe het programma moet zijn opgebouwd. Er zijn verschillende methodieken die een dergelijke werkwijze ondersteunen<sup>1</sup>. Dit heeft aanzienlijke voordelen. Je leert namelijk een moeilijk probleem in te delen in gemakkelijkere deelproblemen (analytische werkwijze). Bovendien heb je tijdens het programmeren van je uitgeschreven ontwerp op papier een houvast. Wat niet onvermeld kan blijven, is dat je tijdens het tentamen ook moet programmeren maar dan met pen op papier. Als je van begin af aan tijdens de practica<sup>2</sup> je opdrachten op papier uitwerkt dan ben je ten tijde van het tentamen aanzienlijk in het voordeel.

### 3. Gebruik testdata

Zorg dat je tijdens het ontwikkelen van je programma testdata tot je beschikking hebt. Als je op papier het programma hebt uitgetekend dan heb je ook eventueel benodigde berekeningen uitgewerkt. Deze berekeningen hebben een resultaat. Jouw programma moet uiteindelijk bij dezelfde invoer dat resultaat geven. Dit stelt je in staat om je programma continu te testen. Zorg altijd voor meerdere sets met testdata!

### 4. Werk systematisch

Programmeer stap voor stap de onderdelen die je op papier hebt uitgewerkt. Bedenk ook hoe je elk onderdeel kunt testen en test dit dan ook. Wij hebben vaker gezien dat de enkele student die van nature systematisch te werk gaat eerder klaar is met de opdrachten en tegelijkertijd minder fouten maakt. Bovendien is systematisch werken goed aan te leren.

### 5. Gebruik de debugger

Als je niet begrijpt waar het programma fout gaat, steek dan niet direct je hand omhoog. Zet in je code op de plek waar het programma vermoedelijk nog correct werkt een *breakpoint*<sup>3</sup>. Als het programma tijdens het debuggen is gestopt op de plek van de breakpoint, controleer dan goed de inhoud van alle actuele variabelen. Dit is één van de krachtigste manieren om fouten te detecteren en om tot een werkend programma te komen.

### 6. Gebruik géén goto-statement

Nog een opmerking moet hier gesteld worden: het gebruik van het goto-statement is niet toegestaan, onder het mom van “gotos are the root of all evil”.

### 7. Gebruik geen spaties en vreemde tekens in bestandsnamen, projectnamen en dergelijke

Windows heeft de onhebbelijke eigenschap om bestandsnamen met spaties toe te staan. Doe het niet! Het leest niet lekker en programma's kunnen niet (goed) functioneren, zeker in combinatie met command line argumenten.

### 8. Maak gebruik van het boek

Het boek kan je vinden op [https://github.com/jesseopdenbrouw/book\\_c](https://github.com/jesseopdenbrouw/book_c). Open de PDF.

Veel succes!

---

<sup>1</sup> Dit komt tijdens de colleges nog aan bod.

<sup>2</sup> En tijdens het maken van huiswerkgaven.

<sup>3</sup> In de les wordt uitgelegd hoe je dit moet doen.

# 1 Inleiding

De practicumdocent zal tijdens het practicum uit hoofdstukken 2 t/m 6 één opdracht aanwijzen die je moet maken. De opdrachten in hoofdstuk 1 moet je allemaal maken.

## 1.1 De gebruikte C-compiler

Wij hebben deze opdrachten uitgewerkt met behulp van Microsoft's **Visual Studio Community**. Visual Studio wordt door zeer veel bedrijven gebruikt. De Community editie is gratis te gebruiken maar je moet wel een Microsoft account aanmaken (of hebben). Een korte installatiehandleiding is te vinden in bijlage A.

Een alternatief is **Code::Blocks**. Code::Blocks is een gratis compiler suite met geïntegreerde invoeromgeving (een zogenoemde IDE = Integrated Development Environment). Hoe je Code::Blocks op je laptop kan installeren kun je vinden in bijlage B. Code::Blocks draait op Windows (7, 8, 8.1, 10), op Linux (Ubuntu, Debian, Fedora Core en CentOS). Er is helaas alleen een oudere versie die op OS-X draait.

Gebruikers van een iMac kunnen ook gebruik maken van **Xcode** die door Apple zelf ontwikkeld is. De functies van Xcode zijn vergelijkbaar met Visual Studio Community en Code::Blocks.

Als je niet de beschikking hebt over een laptop, dan kan je ook online je programma's ontwikkelen. Zie hiervoor de site <http://www.onlinegdb.com/>. Alleen voor de afsluitende opdracht heb je een laptop nodig.

Die-hard Linux-gebruikers kunnen natuurlijk ook in een terminal de GNU-C-compiler direct gebruiken.

## 1.2 Installeren Visual Studio Community en invoeren eerste programma

Voer de handelingen uit in bijlage A. Als je liever Code::Blocks gebruikt, zie bijlage B.

## 1.3 Programma wijzigen

Wijzig de broncode van het programma dat je in de introductie hebt gemaakt als volgt:

---

```
1  /* My first program */
2  #include <stdio.h>
3
4  int main(void) {
5      int a = 6, b = 10
6      printf("Het product van %d en %d is: %d\n", a, b, a + b);
7      printf("\nSluit dit venster door op enter te drukken");
8      getchar();
9      return 0;
10 }
```

---

## 1.4 Vertalen van het gewijzigde programma

Voordat het gewijzigde programma kan worden uitgevoerd moet het programma opnieuw worden vertaald. Je kunt het programma vertalen door op de groene “play” knop te klikken.

Als je het bovenstaande programma juist hebt ingevoerd dan zal bij het vertalen een fout optreden. De foutmelding verschijnt onderin de ontwikkelomgeving in het window “Error list”.

---

```
1  Error C2146 syntax error: missing ';' before identifier 'printf' C:\Users\jesse\source\
   ↪ repos\ep2d\probeer.c 11
```

---

Zie jij wat er fout is?

In de programmeertaal C moet elk “commando” worden afgesloten met een puntkomma en dat is vergeten op de regel voorafgaande aan de regel waarop de foutmelding wordt gegeven. Een fout die er voor zorgt dat een programma niet voldoet aan de regels van de programmeertaal wordt een *syntax fout* genoemd.

Corrigeer de fout en compileer (vertaal) het programma opnieuw.

Als je het bovenstaande programma juist hebt ingevoerd (en de syntax fout hebt gecorrigeerd) dan verschijnt een window met de volgende uitvoer:

---

```
1 Het product van 6 en 10 is: 16
```

---

Sluit dit venster door op enter te drukken.

Zie jij wat er fout is?

Een fout die er voor zorgt dat de uitvoer van een programma niet correct is wordt een *logische fout* genoemd. Logische fouten zijn moeilijker op te sporen dan syntax fouten omdat de compiler logische fouten niet kan vinden. In de praktijk komt het vaak voor dat een logische fout alleen maar onder bepaalde condities optreedt. Als je zelf een programma hebt geschreven dan ben je dus niet klaar als je programma zonder (syntax) fouten compileert. Je moet het programma daarna zelf nog goed testen om logische fouten te voorkomen!

Corrigeer de fout (er zijn twee mogelijkheden!) en test het programma opnieuw.

## 1.5 Programma met invoer

Een integer variabele *a* kan als volgt vanaf het toetsenbord worden ingelezen:

---

```
1 scanf("%d", &a);
```

---

Het zogenoemde *format argument* `"%d"` geeft `scanf` de opdracht om een integer getal in te lezen. Het tweede argument `&a` geeft `scanf` de opdracht om de integer in de variabele *a* op te slaan. Dit wordt in de theorieles nog verder uitgelegd<sup>4</sup>.

Schrijf nu zelf een programma dat twee integers inleest (vanaf het toetsenbord) en de som van deze integers afdruckt.



Let op! De functie `scanf` wordt door Visual Studio bestempeld als *onveilig*. Een alternatief is om gebruik te maken met van de functie `scanf_s` maar niet alle compilers ondersteunen dat. Een andere methode is om `scanf` toch toe te laten met behulp van een *pragma*. Zet aan het begin van het C-bestand de regel:

```
#pragma warning(disable : 4996)
```

Nu kan `scanf` gebruikt worden. Er zijn nog meer functies die deze pragma nodig hebben om te functioneren.

---

<sup>4</sup> Variabele *a* wordt door de computer op een geheugenlocatie opgeslagen. Aan deze geheugenlocatie is een getal gekoppeld, het *adres*. Als `scanf` nu een getal inleest, dan moet dat getal in variabele *a* worden opgeslagen. De `&`-operator geeft het adres van de geheugenlocatie van *a* door aan functie `scanf`.

## 2 Toekennings- en herhalingsopdrachten (for, while)

### 2.1 Inleiding

In de opdrachten van dit hoofdstuk wordt gevraagd een programma te schrijven, waarin gebruik wordt gemaakt van inlees- en schrijfinstructies, toekenningsopdrachten en een herhalingsopdracht m.b.v. een `for` loop.

Als voorbeeld is onderstaand programma gegeven. Hierin worden o.a. wiskundige functies gebruikt. Deze zijn opgenomen in een zogenoemde mathematische bibliotheek. Enkele voorbeelden van functies die in deze bibliotheek zijn gedefinieerd zijn: sinus, cosinus, de vierkantswortel en machtsverheffen. Om deze standaard functies in jouw programma te kunnen gebruiken, zal je in je programma bovenin de volgende *headerfile* moeten opnemen:

---

```
1 #include <math.h>
```

---

De wiskundige functies die in de mathematische bibliotheek voorkomen staan in het boek “De programmeertaal C” in paragraaf 5.11 aangegeven. Je kunt ook online informatie over deze functies vinden op: <http://en.cppreference.com/w/c/numeric/math>

In onderstaand voorbeeldprogramma wordt aan de gebruiker gevraagd een positief getal  $x$  en een getal  $y$  in te voeren. Het programma berekent dan de vierkantswortel uit  $x$  en  $x$  tot de macht  $y$ . Hiervoor wordt gebruik gemaakt van de functies: `sqrt` en `pow`. De `for` loop zorgt ervoor dat vijf keer achter elkaar de vierkantswortel uit  $x$  en  $x$  tot de macht  $y$  wordt berekend. De eerste keer heeft  $x$  de waarde die de gebruiker heeft ingevoerd en de volgende vier keer wordt de waarde van  $x$  met 1 opgehoogd.

Maak een project aan en plaats daarin de onderstaande code. Laat zien dat het werkt.

---

```
1 #include <stdio.h>
2 #include <math.h>
3
4 /* For Visual Studio */
5 #pragma warning(disable : 4996)
6
7 int main(void) {
8     double x, y, wortel, macht;
9     int i;
10    printf("Voer een positief getal x in: ");
11    scanf("%lf", &x);
12    printf("Voer een getal y in: ");
13    scanf("%lf", &y);
14    for (i = 0; i < 5; i = i + 1) {
15        wortel = sqrt(x);
16        macht = pow(x, y);
17        printf("De vierkantswortel uit %f is: %f\n", x, wortel);
18        printf("%f tot de macht %f is: %f\n", x, y, macht);
19        x = x + 1;
20    }
21    getchar();
22    return 0;
23 }
```

---



Let op! Je moet twee van de onderstaande opdrachten maken. De practicumdocent bepaalt welke opdrachten je moet maken.

## 2.2 Kwadraten en derde machten

Schrijf een programma die de kwadraten en derde machten van 1 t/m 10 afdruckt. Gebruik een **for**-statement voor het opzetten van de herhaling.

## 2.3 Tafel van 12

Schrijf een programma dat de tafel van 12 afdruckt. Maak gebruik van een **for**-statement voor het opzetten van de herhaling.

## 2.4 Achterstevoren

Schrijf een programma dat de gebruiker vraagt om een positief geheel getal in te voeren. Druk vervolgens de cijfers van dat getal achterstevoren af. Dus 12345 wordt 54321.

## 2.5 Deelbaar door 9

Schrijf een programma dat alle getallen tussen 100 en 200 afdruckt, die deelbaar zijn door 9. Gebruik een **for**-statement voor het opzetten van de herhaling. Doe dit zonder gebruik te maken van een beslissingsstatement (**if**).

## 2.6 Harmonische reeks

Schrijf een programma dat de inversen van 1 t/m 100 bij elkaar optelt. Dat is dus:

$$\text{som} = \frac{1}{1} + \frac{1}{2} + \frac{1}{3} + \frac{1}{4} + \dots + \frac{1}{100}$$

Gebruik een **for**-statement voor het opzetten van de herhaling.

## 2.7 Driehoek van Floyd

Schrijf een programma dat een positief geheel getal inleest en vervolgens de driehoek van Floyd afdruckt. In deze driehoek worden getallen oplopend afgedrukt, beginnend bij 1, steeds één getal meer per rij. Een voorbeeld van de driehoek voor invoer van het getal 6 is hieronder gegeven. De maximale invoer is 44, dan wordt het getal 999 als hoogste getal afgedrukt.

---

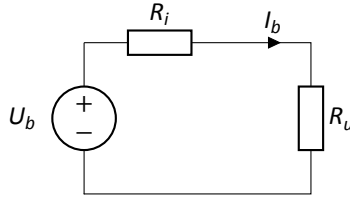
```
1 Geef positief getal: 6
2 1
3 2 3
4 4 5 6
5 7 8 9 10
6 11 12 13 14 15
7 16 17 18 19 20 21
```

---

## 2.8 Inwendig en uitwendig vermogen

Een spanningsbron kan worden voorgesteld door een ideale spanningsbron met de spanning  $U_b$  en een inwendige weerstand  $R_i$ . Aan deze bron wordt een uitwendige weerstand  $R_u$  geplaatst zodat er een stroom  $I_b$  gaat vloeien. Op het moment dat er een stroom vloeit, verbruiken de inwendige weerstand en de uitwendige weerstand vermogen. De bron levert het vermogen. Een voorstelling is te zien in onderstaande figuur.

Ontwerp een programma dat de gebruiker vraagt de waarden van  $U_b$  en  $R_i$  in te voeren. Het programma berekent voor de waarden  $R_u = R_i$  t/m  $R_u = 10 \cdot R_i$  (in stappen van  $R_u$ ) steeds de stroom  $I_b$  en de vermogens



verbruikt in  $R_i$  (de bron) en  $R_u$ . Zorg ervoor dat het programma goed omgaat met  $R_i \leq 0$  door gebruik te maken van een `do while` loop.

Test het programma met  $U_b = 5$  en  $R_i = 10$ . Bij  $R_u = 100$  is  $P_{R_i} = 20,66$  mW en  $P_{R_u} = 206,61$  mW.

Bereken zelf minimaal één set met testwaarden.

## 2.9 Parallelweerstand

De vervangingswaarde van parallel geschakelde weerstanden is uit te rekenen met de formule:

$$\frac{1}{R_p} = \frac{1}{R_1} + \frac{1}{R_2} + \dots + \frac{1}{R_n}$$

waarbij  $R_p$  de vervangingswaarde van de parallel geschakelde weerstanden is en  $R_1, R_2, \dots, R_n$  de waarden van de parallel geschakelde weerstanden zijn.

Ontwerp een programma dat de gebruiker vraagt om het aantal weerstanden in te voeren (groter dan 0 natuurlijk). Daarna worden een voor een de weerstandswaarden ingevoerd. Vervolgens wordt de berekende vervangingswaarde afgedrukt. Zorg ervoor dat bij het invoeren van de gegevens een informatieve tekst wordt afgedrukt zodat de gebruiker weet wat hij/zij moet doen. Voorbeeld:

---

```

1 Geef aantal weerstanden op: 4
2 Geef waarde weerstand 1: 6
3 Geef waarde weerstand 2: 2
4 Geef waarde weerstand 3: 10
5 Geef waarde weerstand 4: 8
6 De vervangingswaarde is: 1.121 Ohm

```

---

## 2.10 Bazel-probleem

Schrijf een programma dat de uitkomst bepaalt van de onderstaande rij:

$$\sum_{i=1}^n \frac{1}{i^2} = \frac{1}{1^2} + \frac{1}{2^2} + \frac{1}{3^2} + \dots + \frac{1}{n^2}$$

De uitkomst is dus de som van de inversen van de kwadraten van 1 tot en met  $n$ . Aan de gebruiker moet  $n$  worden gevraagd. Zorg ervoor dat het ingevoerde getal groter is dan 0 met behulp van een `do while`-lus. Gebruik een `for`-lus om de som te berekenen.

Het is te bewijzen dat als  $n \rightarrow \infty$ , de uitkomst  $\pi^2/6 \approx 1,644934$  bedraagt. Zie <https://nl.wikipedia.org/wiki/Bazel-probleem>.

## 2.11 De spaarrekening

Wanneer je een geldbedrag op een spaarrekening hebt staan en je neemt de daarover uitgekeerde rente niet op, dan groeit het geldbedrag elk jaar met die rente.

Ontwerp een programma dat de gebruiker vraagt het beginbedrag dat op de spaarrekening staat en de rente in procenten die per jaar rente wordt uitgekeerd, in te voeren. Het bedrag staat 10 jaar op de spaarrekening. Het programma moet telkens het bedrag op de spaarrekening aan het eind van het lopende jaar berekenen, en dit bedrag samen met de rente van het betreffende jaar op het scherm afdrukken. Gebruik hiervoor een **for** loop. Zorg ervoor dat voor het beginbedrag en rente alleen positieve getallen kunnen worden ingevoerd door gebruik te maken van een **do while** loop.

Test het programma met: Beginbedrag = 1000 euro, rente = 5%

---

```

1 Geef beginbedrag: 1000
2 Geef rente in procenten: 5
3 jaar   rente   nieuw
4 -----
5   1   50.00   1050.00
6   2   52.50   1102.50
7   3   55.13   1157.63
8   4   57.88   1215.51
9   5   60.78   1276.28
10  6   63.81   1340.10
11  7   67.00   1407.10
12  8   70.36   1477.46
13  9   73.87   1551.33
14 10   77.57   1628.89

```

---

Bereken zelf minimaal nog één set met testwaardes.

## 2.12 De transformator

De spanningen en wikkelingen van de primaire en de secundaire zijde van een transformator verhouden zich als:

$$\frac{U_p}{U_s} = \frac{N_p}{N_s}$$

Hierin is:

- $U_p$  = de spanning aan de primaire zijde;
- $U_s$  = de spanning aan de secundaire zijde;
- $N_p$  = het aantal wikkelingen aan de primaire zijde;
- $N_s$  = het aantal wikkelingen aan de secundaire zijde.

Ontwerp een programma dat de gebruiker vraagt de primaire spanning ( $U_p$ ), het aantal wikkelingen aan de primaire zijde ( $N_p$ ) en de gewenste spanning aan de secundaire zijde ( $U_s$ ) in te voeren. In het programma moet het aantal wikkelingen aan de primaire zijde ( $N_p$ ), vanaf de ingelezen waarde, 10 keer achter elkaar met 100 ophogen. Het programma moet telkens voor elke afzonderlijke  $N_p$ -waarde, beginnend vanaf de ingelezen  $N_p$ -waarde, het aantal benodigde wikkelingen aan de secundaire zijde ( $N_s$ ) berekenen en deze berekende waarde op het scherm afdrukken. Maak gebruik van een **for** loop.

Test het programma met:

---

```

1 Geef primaire spanning Up: 10
2 Geef primaire wikkelingen Np: 10
3 Geef secundaire spanning Us: 200
4   Np   Ns
5   ----
6   10   200
7  110  2200

```

---

|    |      |       |
|----|------|-------|
| 8  | 210  | 4200  |
| 9  | 310  | 6200  |
| 10 | 410  | 8200  |
| 11 | 510  | 10200 |
| 12 | 610  | 12200 |
| 13 | 710  | 14200 |
| 14 | 810  | 16200 |
| 15 | 910  | 18200 |
| 16 | 1010 | 20200 |

---

Bereken zelf minimaal nog één set met testwaardes.

## 2.13 Faculteit

De faculteit van het getal  $n$  wordt genoteerd als  $n!$  en is gedefinieerd als:

$$n! = n \cdot (n - 1) \cdot (n - 2) \cdot \dots \cdot 2 \cdot 1$$

dus voor 5! is dit:

$$5! = 5 \cdot 4 \cdot 3 \cdot 2 \cdot 1$$

Dit is echter ook te schrijven als:

$$5! = 5 \cdot (4 \cdot (3 \cdot (2 \cdot 1)))$$

dus eerst  $2 \cdot 1$  uitrekenen en daarna  $3 \cdot (2 \cdot 1)$  etc.

Ontwerp een programma dat de gebruiker om het gehele getal  $n$  ( $n \geq 0$ ) vraagt en de faculteit  $n!$  uitrekent op basis van herhaald vermenigvuldigen. Zorg ervoor dat het programma ook tussentijdse berekeningen van  $1!$ ,  $2!$ , ...,  $(n - 1)!$  afdruckt. Zorg ervoor dat het programma goed omgaat met de invoer  $n = 0$  en negatieve getallen door gebruik te maken van een **do while** loop.

Bereken zelf minimaal twee sets met testwaardes. Zorg dat je deze testdata bij de hand hebt tijdens het nakijken.

## 2.14 Afgelegde weg

Een kleine stalen kogel wordt onder een hoek  $\alpha$  weggeschoten met een beginsnelheid  $v_0$ . De hoek ligt tussen  $0^\circ$  en  $90^\circ$  in lijn met het aardoppervlakte (de positieve x-as, zoals hoeken altijd getekend worden). De kogel wisselt geen energie uit met de omgeving, de beweging is dus wrijvingsloos. De kogel is natuurlijk ook onder invloed van de zwaartekracht. Dat houdt in dat de kogel een keer op de grond terecht komt. De hoogte en afgelegde weg als functie van de tijd kunnen beschreven worden met de functies:

$$h(t) = v_0 \cdot \sin(\alpha) \cdot t - \frac{1}{2} \cdot g \cdot t^2 \quad [\text{m}]$$

$$s(t) = v_0 \cdot \cos(\alpha) \cdot t \quad [\text{m}]$$

Hierbij  $h(t)$  de hoogte,  $s(t)$  de afgelegde weg ten opzichte van de grond en is  $g$  de gravitatieversnelling, die is  $9,81 \text{ [m/s}^2\text{]}$ . Ontwerp een programma dat de gebruiker om de hoek  $\alpha$  (in graden,  $0^\circ \leq \alpha \leq 90^\circ$ ) en de beginsnelheid  $v_0$  (in km/h,  $v_0 > 0$ ) vraagt en voor de eerste 10 (gehele) seconden de hoogte en de afgelegde weg afdruckt.

Zorg ervoor dat het programma goed omgaat met de invoer  $\alpha < 0^\circ$  en  $\alpha > 90^\circ$  en negatieve waarde voor  $v_0$  door gebruik te maken van een **do while** loop.

Test het programma met:

---

```
1 Geef de hoek in graden (0 - 90): 45
2 Geef de snelheid in km/h (> 0): 200
3 Hoek in radialen: 0.7853982
4 Snelheid in m/s: 55.556
5
6 Tijd      Hoogte [m]    Afstand [m]
7   0        0.00      0.00
8   1       34.38     39.28
9   2       58.95     78.57
10  3       73.71    117.85
11  4       78.65    157.13
12  5       73.79    196.42
13  6       59.12    235.70
14  7       34.64    274.99
15  8        0.35    314.27
```

---

Bereken zelf minimaal één set met testwaardes.

## 2.15 Benadering sinus

De sinus van een hoek wordt benaderd door:

$$\sin x = x - \frac{x^3}{3!} + \frac{x^5}{5!} - \frac{x^7}{7!} + \frac{x^9}{9!} + \dots = \sum_{n=0}^{\infty} \frac{(-1)^n}{(2n+1)!} x^{2n+1}$$

waarbij  $x$  in radialen moet worden opgegeven. De reeks heeft oneindig veel termen, maar een redelijke benadering is al te maken met slechts 5 termen.

Ontwerp een programma dat de gebruiker om een geheel getal  $n$  ( $n \geq 0$ ) vraagt, dit getal stelt het aantal uit te rekenen termen voor. Daarna vraagt het programma om de hoek  $x$  (in radialen). Het programma rekent de sinus uit op basis van herhaald optellen. Maak gebruik een **for** loop. Zorg ervoor dat het programma goed omgaat met de invoer  $n < 0$  door gebruik te maken van een **do while** loop.

Ontwerp een nieuw programma waarbij weer  $n$  en  $x$  worden gevraagd maar waarbij een lijst van uitgerekenende sinuswaarden wordt weergegeven met het aantal termen tussen 0 en  $n$ . Hierbij kan je goed zien hoe de waarde van de sinus van een hoek wordt benaderd. Maak gebruik van een **for** loop.

Bereken zelf minimaal twee sets met testwaardes. Zorg dat je deze testdata bij de hand hebt tijdens het nakijken.

## 2.16 Kerstboom

Een kerstboom is natuurlijk onmisbaar met kerstmis. Om het aantal verbruikte kerstbomen te minderen moet je een programma schrijven dat een kerstboom met '+'-tekens afdruckt. De gebruiker voert een getal  $n$  ( $2 < n < 80$ ) in dat de breedte van de kerstboom voorstelt. Het afdrukken van de kerstboom begint met de top (in het midden) en dijt per afgedrukte regel met twee '+'-tekens uit. De kerstboom moet natuurlijk wel een stam en een 'voet' of standaard hebben. De afmetingen van de stam en de voet zijn (als je het netjes doet) ook afhankelijk van  $n$ . Zorg ervoor dat het programma goed omgaat met de invoer  $n \leq 2$  en  $n \geq 80$  door gebruik te maken van een **do while** loop.

## 2.17 Maandkalender

Schrijf een programma dat een maandkalender afdruckt. De gebruiker moet het aantal dagen in de maand en de dag van de week waarop de maand begint invoeren. Het aantal dagen in de maand kan minimaal 28

en maximaal 31 zijn. De dag van de week waarop de maand begint wordt ingevoerd als een geheel getal met een minimale waarde van 1 (zondag) en een maximale waarde van 7 (zaterdag). Zorg ervoor dat het programma goed omgaat met de invoer van ongeldige getallen.

Voorbeeld van het uitvoeren van dit programma:

---

```

1 Geef het aantal dagen in de maand: 41
2 Geef het aantal dagen in de maand: 31
3 Geef de dag van de week waarop de maand begint (1 = zondag ... 7 = zaterdag): 0
4 Geef de dag van de week waarop de maand begint (1 = zondag ... 7 = zaterdag): 3
5
6 zo ma di wo do vr za
7 =====
8           1  2  3  4  5
9   6  7  8  9 10 11 12
10 13 14 15 16 17 18 19
11 20 21 22 23 24 25 26
12 27 28 29 30 31

```

---

Maak voor het afdrukken van de tabel gebruik van een **for** loop.

## 2.18 Wind-chill temperatuur

Bij het kouder worden van de dagen vragen veel mensen zich af hoe koud het buiten nu eigenlijk aanvoelt. De gevoelstemperatuur is afhankelijk van de omgevingstemperatuur (uit de zon, uit de wind) en de windsnelheid. De gevoelstemperatuur kan als volgt berekend worden:

$$T_{wc} = 13,12 + 0,6215 \cdot T_{omg} - 11,37 \cdot v^{0,16} + 0,3965 \cdot T_{omg} \cdot v^{0,16}$$

De omgevingstemperatuur  $T_{omg}$  is in graden Celsius, de windsnelheid  $v$  is in km/h. De formule geldt alleen voor een omgevingstemperatuur lager dan 10 graden Celsius en een windsnelheid groter dan 4,8 km/h.

Ontwerp een programma dat aan de gebruiker een temperatuur en een windsnelheid vraagt en in een tabel de wind-chill temperatuur afdrukt voor de opgegeven temperatuur en de eerste 9 opvolgende temperaturen (door steeds met 1 graad Celsius te verhogen). Dus als een gebruiker als temperatuur  $-15$  opgeeft, moeten de temperaturen van  $-15$  tot  $-5$  berekend worden.

Zorg ervoor dat de gebruiker alleen temperaturen en windsnelheden invoert die zijn toegestaan (zie eerder in deze opgave).

## 2.19 Benadering e

De wiskundige constante  $e$  wordt benaderd door:

$$e = \frac{1}{0!} + \frac{1}{1!} + \frac{1}{2!} + \frac{1}{3!} + \frac{1}{4!} + \frac{1}{5!} + \dots = \sum_{n=0}^{\infty} \frac{1}{n!}$$

De reeks heeft oneindig veel termen, maar er is al een redelijke benadering te maken met 7 termen. Bij 7 termen is de fout namelijk kleiner dan 0,00005. Ontwerp een programma dat de gebruiker om een geheel getal  $n$  ( $n > 0$ ) vraagt, dit getal stelt het aantal uit te rekenen termen voor. Het programma rekent de constante  $e$  uit op basis van herhaald optellen. Gebruik hiervoor een **for** loop. Zorg ervoor dat het programma goed omgaat met de invoer  $n \leq 0$  door gebruik te maken van een **do while** loop.

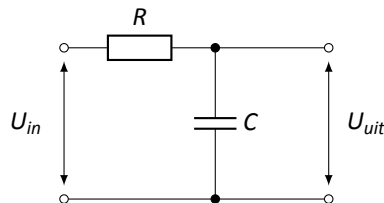
Overigens staat in de formule  $0!$ . De wiskundigen hebben afgesproken dat  $0!$  gelijk is aan 1. Bij het gebruik van integers kan  $n$  niet te groot zijn (maximaal 13) omdat bij grotere waarden overflow in het berekenen van de faculteit optreedt. Je kan ook gebruik maken van doubles.

Ontwerp een nieuw programma waarbij weer  $n$  wordt gevraagd maar waarbij een lijst van uitgerekende benaderingen van de constante  $e$  wordt weergegeven met het aantal termen tussen 1 en  $n$ . Hierbij kan je goed zien hoe de waarde van de constante  $e$  wordt benaderd. Maak gebruik van een **for** loop.

Controleer de werking van het programma met:  $e = 2.718281828$

## 2.20 Opladen van een condensator

In de onderstaande figuur is een schakeling met een weerstand  $R$  en een condensator  $C$  te zien. Alle spanningen in de schakeling zijn in de beginsituatie 0 V.



Op een bepaald moment wordt op de ingang een constante spanning van 10 V aangeboden. De condensator zal hierdoor langzaam “vollopen” waardoor de spanning over de condensator stijgt. De snelheid waarmee dat gebeurt, is afhankelijk van de waarden van  $R$  in Ohm,  $C$  in Farad en de tijd  $t$  in seconden. De uitgangsspanning  $U_{uit}$  als functie van de tijd kan als volgt berekend worden:

$$U_{uit}(t) = 10 \cdot (1 - e^{-t/RC}) \quad (e \text{ is het grondtal van de natuurlijke logaritme, } e \approx 2,71828)$$

Schrijf een programma dat de uitgangsspanning  $U_{uit}$  berekend als functie van  $R$ ,  $C$  en de tijd. Het programma moet de waarden van  $R$  en  $C$  inlezen (beide moeten groter zijn dan 0) en daarna moet het aantal te berekenen seconden worden ingevoerd (groter dan 0). Het programma berekent vanaf tijdstip 0 op iedere seconde de waarde van de uitgangsspanning. Hieronder zie je een mogelijke uitvoer van het programma met de berekende spanning op twee decimalen nauwkeurig.

---

```

1 Geef de waarde van R (R > 0): 2.5
2 Geef de waarde van C (C > 0): 1.4
3 Geef het aantal seconden op (t > 0): 20
4 tijd spanning
5 -----
6 0 0.00
7 1 2.49
8 2 4.35
9 3 5.76
10 4 6.81
11 5 7.60
12 6 8.20
13 7 8.65
14 8 8.98
15 9 9.24
16 10 9.43
17 11 9.57
18 12 9.68
19 13 9.76
20 14 9.82
21 15 9.86
22 16 9.90
23 17 9.92
24 18 9.94
25 19 9.96

```

---

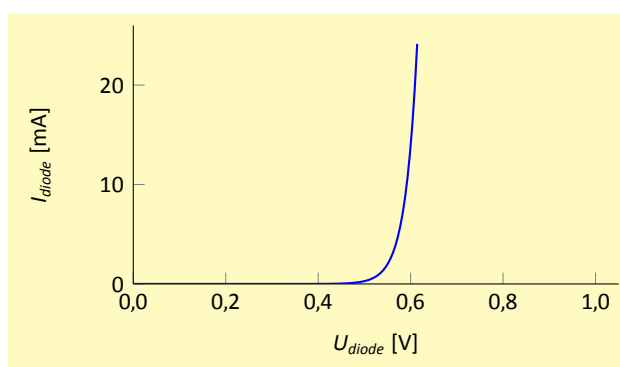
Merk op dat, naar mate de tijd toeneemt, de uitgangsspanning steeds dichterbij 10 V komt, maar nooit helemaal 10 V zal worden. (Uiteindelijk wel, maar dat heeft met afrondingen van de floating point-getallen te maken).

## 2.21 Stroom door een diode

Een *diode* is een elektronische component die vaak gebruikt wordt, bijvoorbeeld in gelijkrichtschakelingen. De relatie tussen de spanning *over* en de stroom *door* de diode is gegeven met de formule van Shockley en geldt voor een ideale diode:

$$I_{diode} = I_s \cdot (e^{\frac{U_{diode}}{U_T}} - 1) \quad (e \text{ is het grondtal van de natuurlijke logaritme, } e \approx 2,71828)$$

Hierin is  $I_s$  de donkerstroom (typisch  $1 \times 10^{-9}$  mA) en  $U_T$  de thermische spanning (typisch 25 mV).

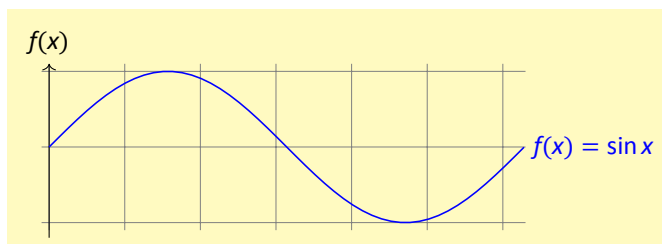


We zien dat de stroom nagenoeg 0 is bij spanningen kleiner dan 0,4 V en erg groot is bij spanningen groter dan 0,6 V. Daartussen stijgt de stroom zeer snel, en dat is het gebied dat onderzocht moet worden.

Schrijf een programma dat de stroom door de diode berekent, tussen de spanningen 0,4 V en 0,6 V met stappen van 0,01 V.

## 2.22 Booglengte van één periode van een sinus

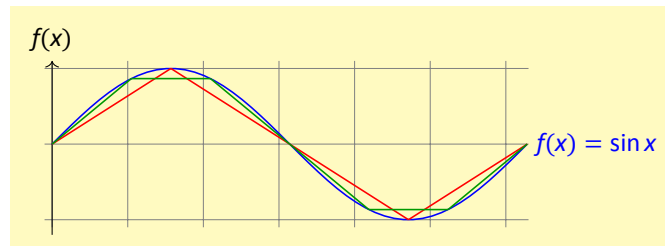
We willen graag de booglengte weten van één periode van een sinus. We gaan uit van de functie  $f(x) = \sin x$ . Eén periode van de sinus is getekend in de onderstaande figuur.



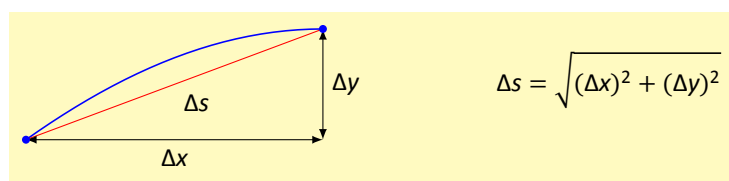
Nu is de booglengte  $s$  exact uit te rekenen met de integraal:

$$s = \int_0^{2\pi} \sqrt{1 + \cos^2 x} \, dx$$

Helaas is deze integraal niet analytisch oplosbaar. Er zit niets anders op om de lengte numeriek uit te rekenen. Daartoe hanteren we het volgende. We delen de sinus op (via de x-as) in een aantal stukken. In de onderstaande figuur is te zien dat de sinus in vier stukken is opgedeeld (rode lijn) en in zes stukken (groene lijn). De lengte van de rode en groene lijnen benaderen de booglengte van de sinus aardig.



De lengte van één stuk van de rode lijn is uit te rekenen met behulp van de stelling van Pythagoras, zoals te zien is in de onderstaande figuur. Hierbij staat het teken  $\Delta$  voor “een stukje lengte”.



Als we nu alle lengtes van de lijnstukken optellen, benaderen we de booglengte van de sinus. Als we nu het aantal stukken steeds groter maken, benaderen we de booglengte van de sinus steeds beter.

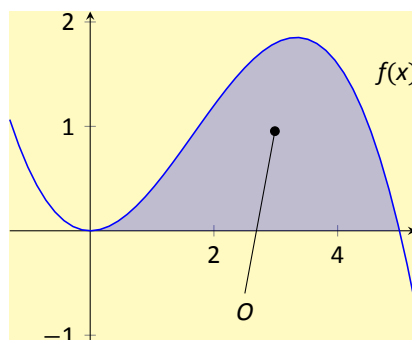
Schrijf een programma dat de lengte van één periode van de sinus bepaalt. Het aantal stappen moet worden ingevoerd als een geheel getal  $n$  ( $n > 1$ ).

## 2.23 Oppervlakte bepalen

Gegeven is de functie:

$$f(x) = -\frac{1}{10}x^3 + \frac{1}{2}x^2$$

De functie is te zien in onderstaande figuur voor de grenzen  $x = 0$  tot  $x = 5$ .



Schrijf een programma dat de oppervlakte van de functie berekent binnen het domein  $x = 0$  tot en met  $x = 5$ . Dit is in de figuur aangegeven met de lichtblauwe kleur. De oppervlakte is aangegeven met de letter  $O$ . Tip: lees het boek. Overigens is het antwoord exact  $125/24 \approx 5,2083 \dots$

## 3 Beslissingsopdrachten (if, if else, switch)

### 3.1 Inleiding

In de opdrachten van dit hoofdstuk wordt gevraagd een programma te schrijven, waarin gebruik wordt gemaakt van inlees- en schrijfinstructies, toekenningsopdrachten, een herhalingsopdracht en één of meer beslissingsopdrachten. Er zijn drie verschillende beslissingsopdrachten: **if**, **if else** en **switch**. De **if** of **if else** kun je gebruiken bij een eenvoudige beslissing, bijvoorbeeld om te beslissen of een getal groter is dan 10. De **switch** gebruik je als je één variabele moet testen op verschillende mogelijke waarden. Hieronder zie je een voorbeeld van het gebruik van een **switch**.

Maak een project aan en plaats daarin de onderstaande code. Laat zien dat het werkt.

```
1 #include <stdio.h>
2
3 /* For Visual Studio */
4 #pragma warning(disable : 4996)
5
6 int main(void){
7     int cijfer;
8     char letter;
9     do {
10         printf("Geef je cijfer: ");
11         scanf("%d", &cijfer);
12     } while (cijfer < 0 || cijfer > 10);
13
14     switch (cijfer) {
15         case 10:
16         case 9:
17         case 8:
18             letter = 'A'; break;
19         case 7:
20             letter = 'B'; break;
21         case 6:
22             letter = 'C'; break;
23         case 5:
24             letter = 'D'; break;
25         default:
26             letter = 'F'; break;
27     }
28     printf("Dit komt overeen met een %c.\n", letter);
29     getchar();
30     return 0;
31 }
```

Let bij het **switch**-statement gebruik wordt gemaakt van het **break**-statement. Hierdoor wordt de **switch** direct verlaten. Soms is het nodig om meerdere constanten samen te nemen. Dan moet je geen **break**-statement gebruiken. Zie het voorbeeld.

Een **switch** alleen interessant als je moet testen op meerdere mogelijkheden van één variabele. Een **if** is veel veelzijdiger.



Let op! Je moet twee van de onderstaande opdrachten maken. De practicumdocent bepaalt welke opdrachten je moet maken.

### 3.2 Grootste getal

Schrijf een programma dat drie gehele getallen inleest het grootste van de drie getallen afdruckt.

### 3.3 Een na grootste getal

Schrijf een programma dat het op één na grootste getal bepaalt uit een aantal opgegeven *gehele* getallen. Vraag eerst aan de gebruiker om hoeveel getallen het gaat.

### 3.4 Kleinste en één na kleinste getal

Schrijf een programma dat het kleinste en één na kleinste getal bepaalt uit een aantal opgegeven *gehele* getallen. De rij wordt afgesloten met het getal 0.

### 3.5 Naam van de maand afdrukken

Ontwerp een programma dat de gebruiker vraagt om positieve gehele getallen tussen 1 en 12 (inclusief) in te voeren en de bijbehorende naam van de maand af te drukken. Dit programma moet net zo lang getallen inlezen totdat het getal 0 wordt ingevoerd. Dan moet het programma stoppen. Bij invoeren van een te groot of te klein getal moet "onbekende maand" afgedrukt worden.

### 3.6 Cijfer als woord afdrukken

Schrijf een programma dat een cijfer (0 t/m 9) van het toetsenbord inleest en dit cijfer als woord afdruckt. Dit moet het programma net zo lang doen tot het getal -1 is ingevoerd. Elke andere combinatie moet leiden tot het woord "onbekend cijfer".

### 3.7 Verandering van ingevoerde getallen

Ontwerp een programma dat aangeeft of een ingevoerd geheel getal *veranderd* is ten opzichte van het vorige ingevoerde getal. Let op: het eerste ingevoerde getal heeft geen voorganger dus kan niet bepaald worden of het veranderd is. Het getal 0 zorgt ervoor dat het programma stopt.

### 3.8 Even en oneven

Ontwerp een programma dat de gebruiker vraagt om positieve gehele getallen in te voeren. Dit programma moet net zo lang getallen inlezen totdat het getal 0 wordt ingevoerd. Hierna moet het programma afdrukken hoeveel even en hoeveel oneven getallen door de gebruiker zijn ingevoerd. Als de gebruiker negatieve getallen invoert dan moeten die niet worden meegeteld. Bedenk goed met welke invoerreeksen je dit programma moet testen om de correcte werking aan te tonen. Laat de invoerreeksen die je hebt gebruikt ook aan je practicumdocent zien.

Tip: Een even getal is deelbaar door 2 en een oneven getal is niet deelbaar door 2. Als een getal  $x$  deelbaar is door 2 dan is  $x \% 2$  gelijk aan 0. Als een getal  $y$  niet deelbaar is door 2 dan is  $y \% 2$  gelijk aan 1. Zie het boek voor uitleg over de %-operator.

Verzin zelf een aantal testreeksen. Zorg dat je deze testdata bij de hand hebt tijdens het nakijken.

### 3.9 Voldoende en onvoldoende

Ontwerp een programma dat de gebruiker vraagt om positieve gehele getallen in te voeren met een maximum van 10. Dit programma moet net zo lang getallen inlezen totdat het getal 0 wordt ingevoerd. Hierna moet het programma afdrukken hoeveel voldoende (6...10) en hoeveel onvoldoendes (1...5) door de gebruiker zijn ingevoerd. Als de gebruiker negatieve getallen of getallen groter dan 10 invoert dan moeten die door het programma niet worden meegeteld. Bedenk goed met welke invoerreeksen je dit programma moet testen om de correcte werking aan te tonen. Laat de invoerreeksen die je hebt gebruikt ook aan je practicumdocent zien.

Verzin zelf een aantal testreeksen. Zorg dat je deze testdata bij de hand hebt tijdens het nakijken.

### 3.10 Aantal ingevoerde cijfers afdrukken

Schrijf een programma dat een reeks ingevoerde cijfers (0 t/m 9) van willekeurige lengte van het toetsenbord inleest en bijhoudt hoe vaak elk cijfer is ingevoerd. De invoer wordt afgesloten met een enter-toets. Daarna moeten de aantallen van de ingevoerde cijfers netjes worden afgedrukt. Hint: lees karakters in.

### 3.11 PIN-code raden

Schrijf een programma dat de gebruiker vraagt om een 4-cijferige PIN-code correct in te voeren. De gebruiker mag maximaal drie keer invoeren. Als na de derde keer een verkeerde PIN-code is ingevoerd, moet een foutmelding op het scherm volgen. Als de PIN-code correct is ingevoerd, moet dat natuurlijk gemeld worden.

Voor het genereren van een willekeurig getal tussen 0 en 9999, kan de functie `rand` gebruikt worden. Zie voor een eenvoudige uitleg <https://www.geeksforgeeks.org/rand-and-srand-in-cpp/>.

### 3.12 Overwerk

Overwerk kan vervelend zijn maar in veel gevallen levert het extra salaris op. De nominale werkweek in Nederland is 40 uur. Daarboven wordt (vaak) gesproken over overwerk. Tussen 40 uur en 50 uur wordt gesproken van matig overwerk, boven de 50 uur wordt gesproken van veel overwerk.

Schrijf een programma dat het salaris van een werkweek bepaalt. Eerst wordt aan de gebruiker gevraagd wat het standaard tarief is van één uur werk. Alle uren tot 40 uur worden met dit tarief berekend. De uren die tussen 40 uur en 50 uur worden gewerkt leveren 1,5 keer het 40-uurs-tarief op. Uren boven de 50 uur leveren 2 maal het 40-uurs-tarief op. Zorg ervoor dat alleen een positief aantal uren en uurtarief kan worden opgegeven.

### 3.13 ABC-formule

Schrijf een programma dat de wortels van een kwadratische vergelijking bepaalt. Een kwadratische vergelijking heeft de gedaante:

$$ax^2 + bx + c = 0 \quad (a \neq 0)$$

De wortels van de functie, dat zijn de waarden van  $x$  waarvoor geldt dat het linkerlid 0 oplevert, zijn te berekenen met de ABC-formule:

$$x_1 = \frac{-b + \sqrt{b^2 - 4ac}}{2a} \quad \text{en} \quad x_2 = \frac{-b - \sqrt{b^2 - 4ac}}{2a}$$

De term  $b^2 - 4ac$  onder het wortelteken staat bekend als de *discriminant*. Deze discriminant moet een waarde hebben die groter is dan of gelijk is aan 0. Een negatieve waarde geeft aan dat er geen oplossingen zijn voor de wortels. Merk op dat als de discriminant 0 is, de waarden van  $x_1$  en  $x_2$  aan elkaar gelijk zijn.

Schrijf een programma dat de gebruiker om de waarden van  $a$ ,  $b$  en  $c$  vraagt en de waarden van  $x_1$  en  $x_2$  berekent.

### 3.14 Simpele rekenmachine

Rekenen doen we al sinds mensenheugenis. Vooral in de elektrotechniek is rekenen onontbeerlijk. We gebruiken daarbij een rekenmachine met veel functies. Schrijf een programma dat een zeer eenvoudige

rekenmachine implementeert. De rekenmachine kan slechts optellen, aftrekken, vermenigvuldigen en delen. Het programma vraagt de gebruiker om op één regel de twee getallen, gescheiden door een operator (\*, +, – en /), op te geven. Voorbeeld:

---

```
1 Geef twee getallen gescheiden door een operator: 5.2 * 4.5
2 De vermenigvuldiging van 5.2 en 4.5 is 23.4
```

---

Maak gebruik van het `switch`-statement. Zorg voor correcte foutafhandeling.

### 3.15 Afdrukken binair getal

Het afdrukken van een binair getal is in standaard C niet mogelijk. Natuurlijk zijn er uitbreidingen in C-compilers, maar daar kan je niet van uitgaan. Ontwerp een programma dat een geheel getal tussen 0 en 255 omzet naar een 8-bits binair getal. Bedenk goed met welke getallen je dit programma moet testen om de correcte werking aan te tonen. Laat de getallen die je hebt gebruikt om te testen ook aan je practicumdocent zien.

Hint: om te kijken of het hoogste bit (bit 7; we nummeren van 0 t/m 7) een 0 of 1 is, kan je het getal vergelijken met 128, de hoogste macht in een 8-bit binair getal. Is het getal 128 of groter, dan is het meest significante bit 1, anders is het 0. Zo kan je alle bits afgaan. Gebruik een `for`-lus om de oplossing efficiënt te programmeren.

Zorg ervoor dat het aantal af te drukken bits eenvoudig gewijzigd kan worden. Neem in je code bijvoorbeeld de variabele `aantal_bits` op:

---

```
1 int aantal_bits = 16;
```

---

om een ingevoerd getal met 16 bits af te drukken.

Vind zelf een aantal testwaardes.

### 3.16 Windsnelheden

Varen op de wereldse zeeën is iets wat de Nederlanders al generaties doen. Om een goed beeld te hebben van de kracht van de wind is rond 1805 de zogenaamde Beaufort-schaal ingevoerd (Bf). Tegenwoordig werken we meer met snelheden in km/h. Toch wordt er nog veel met Beaufort gewerkt.

Ontwerp een programma dat de gebruiker vraagt om een windsnelheid in km/h (geheel getal). Het programma moet dan de windsnelheid in Beaufort afdrukken. Het programma moet dit steeds herhalen. Het programma stopt als het getal –1 wordt ingevoerd als windsnelheid. Bedenk goed met welke getallen je dit programma moet testen om de correcte werking aan te tonen. Laat de getallen die je hebt gebruikt om te testen ook aan je practicumdocent zien.

Je kan een conversietabel vinden op <https://keesvdm.home.xs4all.nl/boot/windkracht.htm>

Hou de testwaardes bij de hand voor tijdens het nakijken van de opdracht.

### 3.17 Geschreven letters

Ontwerp een programma dat een geheel getal tussen 0 en 100 (inclusief) inleest en dit getal daarna als woord afdruckt. Als voorbeeld: de invoer 21 levert als uitvoer `eenentwintig`. Het programma blijft dit doen totdat het getal –1 wordt ingevoerd. Ontwerp dit programma zo efficiënt mogelijk. Bedenk goed met welke getallen je dit programma moet testen om de correcte werking aan te tonen. Laat de getallen die je hebt gebruikt om te testen ook aan je practicumdocent zien.

Test het programma met:

---

```
1 Geef getal tussen 0 en 100: 21
2 Het getal is: eenentwintig
```

---

Geef zelf ook een aantal testwaardes. Zorg dat je deze testdata bij de hand hebt tijdens het nakijken.

### 3.18 Stijgen

Ontwerp een programma dat de gebruiker vraagt om positieve gehele getallen in te voeren. Dit programma moet net zo lang getallen inlezen totdat het getal 0 wordt ingevoerd. Op dat moment moet het programma afdrukken hoe vaak de invoer van de gebruiker “gestegen” is bij het invoeren. Als de gebruiker een getal invoert dat groter is dan zijn/haar vorige invoer dan tellen we dat als een stijging. Als de gebruiker negatieve getallen invoert dan moeten deze na het inlezen worden overgeslagen. Bedenk goed met welke invoerreeksen je dit programma moet testen om de correcte werking aan te tonen. Laat de invoerreeksen die je hebt gebruikt ook aan je practicumdocent zien.

Tip: in de volgende tabel zijn enkele invoerreeksen met de gewenste uitvoer gegeven. Test of je programma werkt door de onderstaande reeksen in te voeren.

| Invoer                             | Correcte uitvoer | Verklaring                                                                                            |
|------------------------------------|------------------|-------------------------------------------------------------------------------------------------------|
| 10 10 20 19 19 21 0                | 2                | Bij het invoeren van 20 na 10 is er een stijging en bij het invoeren van 21 na 19 is er een stijging. |
| -1 10 -1 10 -1 -1 20 -2 19 19 21 0 | 2                | Als we alle negatieve getallen overslaan dan is deze invoer hetzelfde als de vorige.                  |

Verzin zelf een aantal testreeksen. Zorg dat je deze testdata bij de hand hebt tijdens het nakijken.

### 3.19 Tekens tellen

Ontwerp een programma dat een reeks tekens inleest en de tekens classificeert als letters, cijfers en leestekens. Het aantal ingevoerde letters, cijfers en leestekens moeten worden afgedrukt. Bij letters moet nog onderscheid worden gemaakt tussen klinkers en medeklinkers. Bedenk goed met welke invoerreeksen je dit programma moet testen om de correcte werking aan te tonen. Laat de invoerreeksen die je hebt gebruikt ook aan je practicumdocent zien. Leestekens zijn te vinden op <http://nl.wikipedia.org/wiki/Leesteken>.

Tip: het testen van het type van een teken kan heel eenvoudig met de karakter classificatie functies uit de standaard C library, zie <http://en.wikipedia.org/wiki/Ctype.h>.

Verzin zelf een aantal testreeksen. Zorg dat je deze testdata bij de hand hebt tijdens het nakijken.

### 3.20 Dag van de week

Ontwerp een programma dat de gebruiker vraagt om datums in het formaat *dd mm yyyy* (dag gevolgd door maand gevolgd door jaartal, alle in cijfers) in te voeren. Deze datum getest worden op geldigheid. Daarna moet het programma de dag van de week voor deze datum bepalen. Er moet uiteraard getest worden of een gebruiker een geldige datum heeft ingevoerd. Het programma herhaalt het hiervoor beschreven proces totdat de speciale datum 00 00 0000 ingevoerd wordt. Dan moet het programma stoppen. Maak gebruik van een *switch*-statement. Bedenk goed met welke datums je dit programma moet testen om de correcte werking aan te tonen. Laat de datums die je hebt gebruikt om te testen ook aan je practicumdocent zien.

Een algoritme voor het vinden van de weekdag van een datum is te vinden op [https://en.wikipedia.org/wiki/Determination\\_of\\_the\\_day\\_of\\_the\\_week#Sakamoto's\\_methods](https://en.wikipedia.org/wiki/Determination_of_the_day_of_the_week#Sakamoto's_methods)

Zoek zelf testdata. Zorg dat je deze testdata bij de hand hebt tijdens het nakijken.

### 3.21 Bepalen schrikkeljaar

Ontwerp een programma dat de gebruiker vraagt om datums in het formaat *dd mm yyyy* (dag gevolgd door maand gevolgd door jaartal, alle in cijfers) in te voeren. Deze datum getest worden op geldigheid. De

datum moet opnieuw worden afgedrukt, maar nu moet de maand als woord worden afgedrukt. Daarnaast moet het programma afdrukken welke dag in het jaar het is en of het een schrikkeljaar is. De invoer 24 11 2011 levert dus op: 24 november 2011, dag 328 van het jaar, geen schrikkeljaar. Er moet uiteraard getest worden of een gebruiker een geldige datum heeft ingevoerd. Het programma herhaalt het omzetten totdat de speciale datum 00 00 0000 ingevoerd wordt. Dan moet het programma stoppen. Maak gebruik van een `switch`-statement. Bedenk goed met welke datums je dit programma moet testen om de correcte werking aan te tonen. Laat de datums die je hebt gebruikt om te testen ook aan je practicumdocent zien.

Het bepalen of een bepaald jaar en schrikkeljaar is kan je vinden op <http://www.daniweb.com/software-development/c/threads/233325>

Vind zelf een aantal testdata. Zorg dat je deze testdata bij de hand hebt tijdens het nakijken.

### 3.22 Extra dagen

Schrijf een programma dat vraagt om een geldige datum in het formaat `dd mm yyyy` (dag gevolgd door maand gevolgd door jaartal, alle in cijfers). Deze datum getest worden op geldigheid. Daarna wordt gevraagd om een extra aantal dagen (alleen dagen). Deze dagen moeten worden opgeteld bij de ingevoerde datum. Druk vervolgens deze nieuwe datum af. Let erop dat de nieuwe datum geldig moet zijn.

### 3.23 Stelsel van twee lineaire vergelijkingen

Gegeven is het stelsel van twee lineaire vergelijkingen:

$$a_1x + b_1y = c_1$$

$$a_2x + b_2y = c_2$$

waarbij  $a_1$ ,  $b_1$ ,  $c_1$ ,  $a_2$ ,  $b_2$  en  $c_2$  constanten zijn (geheel of floating point).

Schrijf een programma dat de gegevens  $a_1$ ,  $b_1$ ,  $c_1$ ,  $a_2$ ,  $b_2$  en  $c_2$  van het toetsenbord inleest en vervolgens het stelsel oplost. Hint: bereken eerst  $d = a_1b_2 - a_2b_1$ . Als  $d = 0$  dan is er geen oplossing voor dit stelsel. Als  $d \neq 0$ , kunnen  $x$  en  $y$  berekend met:

$$x = \frac{b_2c_1 - b_1c_2}{d}$$

$$y = \frac{a_1c_2 - a_2c_1}{d}$$

Opmerking: een stelsel van lineaire vergelijkingen komt voor in het oplossen van *knooppuntspanningen* in een elektrisch schema.

## 4 Functies

### 4.1 Inleiding

Deze opdracht gaat over het gebruik van functies. Veel voorkomende handelingen, zoals het uitrekenen van verschillende waarden van formules of het inlezen van getallen kunnen hiermee efficiënt gecodeerd worden. Daarnaast wordt het gehele programma overzichtelijker. Bepaalde algoritmes, zoals het berekenen van een faculteit of het sorteren van een rij getallen, kunnen elegant worden opgelost door het gebruik van *recursive functies* (functies die zichzelf aanroepen). De functies moeten grondig getest worden. Dit kan natuurlijk in hetzelfde programma als waar de functie zich in bevindt.

Na de laatste theorieles in week 3 heb je de huiswerkopdracht gekregen om een recursieve functie te schrijven die het  $n^e$  getal uit de *Fibonacci-rij* ([https://nl.wikipedia.org/wiki/Rij\\_van\\_Fibonacci](https://nl.wikipedia.org/wiki/Rij_van_Fibonacci)) berekent. Een mogelijke oplossing is:

```
1 #include <stdio.h>
2
3 /* For Visual Studio */
4 #pragma warning(disable : 4996)
5
6 int fib(int n) {
7     if (n < 2) {
8         return n;
9     }
10    else {
11        return fib(n-1) + fib(n-2);
12    }
13 }
14
15 int main(void) {
16     int getal;
17
18     do {
19         printf("Geef een getal groter dan of gelijk aan 0: ");
20         scanf("%d", &getal);
21     } while (getal < 0);
22
23     printf("fib(%d) is %d\n", getal, fib(getal));
24
25     printf("Druk op enter om het programma af te sluiten.");
26     getchar();
27     return 0;
28 }
```

Maak een project aan en plaats daarin de bovenstaande code. Laat zien dat het werkt.

Let erop dat de functie `fib()` een *stopcriterium* moet hebben, d.w.z. dat het herhaald aanroepen van `fib`  $\hookrightarrow$  `()` (recursie) stopt als aan een bepaalde voorwaarde wordt voldaan. In het bovenstaande programma is dat als variabele  $n$  kleiner is dan twee. Vervang eens voor de grap de functie `fib()` met:

```
1 int fib(int n) {
2     return fib(n-1) + fib(n-2);
3 }
```



Let op! Je moet twee van de onderstaande opdrachten maken. De practicumdocent bepaalt welke opdrachten je moet maken.

## 4.2 Vermenigvuldigen

Schrijf een functie die twee argumenten meekrijgt en de vermenigvuldiging van deze twee argumenten teruggeeft. Schrijf een hoofdprogramma om de functie te testen.

## 4.3 Vermenigvuldigen en delen

Schrijf een functie die drie argumenten meekrijgen en het resultaat van de vermenigvuldiging van de eerste twee argumenten gedeeld door het derde argument teruggeeft. Let op dat delen door 0 niet kan. Zorg dat het programma hier goed mee omgaat. Schrijf een hoofdprogramma om de functie te testen.

## 4.4 Catalan-getal (iteratief)

De Catalan-getallen zijn natuurlijke getallen die voorkomen in diverse telproblemen. Het  $n^e$  Catalan-getal wordt weergegeven als  $C_n$ . Bijvoorbeeld: het aantal manieren waarop je  $n$  paren haakjes correct kunt opschrijven is gelijk aan  $C_n$ . Zie voor verdere informatie: [https://en.wikipedia.org/wiki/Catalan\\_number](https://en.wikipedia.org/wiki/Catalan_number).

$C_n$  kan berekend worden met behulp van de faculteitsfunctie:

$$C_n = \frac{(2n)!}{(n+1)!n!}$$

- Implementeer een functie `catalan` die de waarde van  $C_n$  berekent met behulp van de bovenstaande formule. Maak daarbij gebruik van de in de les behandelde functie `faculteit` die  $n!$  berekent.
- Schrijf een testprogramma om deze functie grondig te testen.
- Test tot welke waarde van  $n$  de functie `catalan` de waarde van  $C_n$  correct berekent.

Test het programma met de volgende invoer.

Voer 0 in, uitvoer is 1  
Voer 5 in, uitvoer is 42  
Voer 7 in, uitvoer is 429  
Voer 9 in, uitvoer is 4862  
Voer 10 in, uitvoer is 16796

## 4.5 Catalan-getal (recursief)

De Catalan-getallen zijn natuurlijke getallen die voorkomen in diverse telproblemen. Het  $n^e$  Catalan-getal wordt weergegeven als  $C_n$ . Bijvoorbeeld: het aantal manieren waarop je  $n$  paren haakjes correct kunt opschrijven is gelijk aan  $C_n$ . Zie voor verdere informatie: [https://en.wikipedia.org/wiki/Catalan\\_number](https://en.wikipedia.org/wiki/Catalan_number).

$C_n$  kan ook recursief berekend worden:

$$C_n = \frac{2(2n-1)}{n+1} C_{n-1} \text{ voor } n \geq 1 \text{ en } C_0 = 1$$

- Implementeer een functie `catalanRecursief` die de waarde van  $C_n$  berekent met behulp van de bovenstaande recursieve formule.
- Schrijf een testprogramma om deze functie grondig te testen.
- Test tot welke waarde van  $n$  de functie `catalanRecursief` de waarde van  $C_n$  correct berekent.

Test het programma met de volgende invoer.

Voer 0 in, uitvoer is 1  
Voer 5 in, uitvoer is 42  
Voer 7 in, uitvoer is 429  
Voer 9 in, uitvoer is 4862  
Voer 10 in, uitvoer is 16796

## 4.6 Vermoeden van Collatz

Het vermoeden van Collatz is één van de bekende onopgeloste vraagstukken in de getaltheorie. Laat  $n$  ( $n > 0$ ) een willekeurig geheel getal zijn. Definieer een functie  $f_n$  als:

$$f_n = \begin{cases} n/2 & \text{als } n \text{ even is} \\ 3n + 1 & \text{als } n \text{ oneven is} \end{cases}$$

Het vermoeden van Collatz zegt dat als we deze formule *herhaaldelijk* toepassen uiteindelijk altijd de waarde 1 bereikt wordt voor elke beginwaarde van  $n$ . Zie voor verdere informatie: [https://en.wikipedia.org/wiki/Collatz\\_conjecture](https://en.wikipedia.org/wiki/Collatz_conjecture).

- Implementeer een functie `int f(int n)` die de waarde van  $f_n$  berekent met behulp van de bovenstaande formule.
- Schrijf een testprogramma om deze functie grondig te testen.
- Ontwerp en implementeer nu een functie `int collatzSteps(int n)` die voor een bepaalde waarde van  $n$  bijhoudt hoe vaak de functie `f` aangeroepen wordt voordat de waarde 1 wordt bereikt.
- Schrijf een testprogramma om deze functie grondig te testen.

Bij grote waarden van  $n$  kan er een fout optreden bij het berekenen van `collatzSteps` omdat de waarde van  $f_n$  niet in een integer variabele past.

Bepaal tot welke waarde van  $n$  de functie `collatzSteps` het aantal stappen correct berekent.

Tip: Je kunt de functie `f` aanpassen zodat de waarde  $-1$  wordt teruggegeven als het resultaat niet in een integer past. Bedenk zelf hoe je dit kunt detecteren als gegeven is dat (op een PC) de grootste waarde die in een integer past  $2^{31} - 1$  is. Vervolgens kun je dan de functie `collatzSteps` aanpassen zodat de waarde  $-1$  wordt teruggegeven als `f` de waarde  $-1$  teruggeeft. Vervolgens kun je `collatzSteps` aanroepen voor opeenvolgende waarden van  $n$  totdat  $-1$  wordt teruggegeven om te achterhalen tot welke waarde van  $n$  `collatzSteps` correct werkt.

Een mogelijke uitvoering van het programma zie je hieronder:

---

```
1 Geef geheel getal groter dan 1: 6
2 Stap: 1, n = 3
3 Stap: 2, n = 10
4 Stap: 3, n = 5
5 Stap: 4, n = 16
6 Stap: 5, n = 8
7 Stap: 6, n = 4
8 Stap: 7, n = 2
9 Stap: 8, n = 1
10 Aantal stappen voor 6 is 8
```

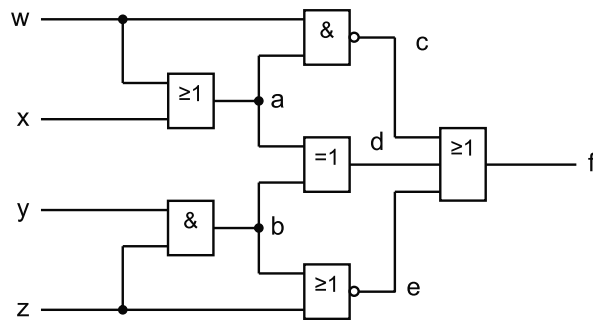
---

Bereken zelf ook een aantal testwaardes. Zorg dat je deze testdata bij de hand hebt tijdens het nakijken.

## 4.7 Digitale techniek

In de digitale wordt gebruik gemaakt van de bekende basispoorten AND, OR, NOT, NAND en NOR. Daarnaast worden ook de veel voorkomende poorten EXOR en EXNOR gebruikt, alhoewel dit geen basispoorten zijn.

Een zekere docent Digitale Techniek wil testen of zijn studenten een digitale schakeling met basispoorten correct kunnen doorrekenen, maar natuurlijk wil hij zelf niet teveel rekenwerk doen. Daarom wil hij een programma schrijven om dit doorrekenwerk te versnellen. Helaas heeft hij door de drukke werkzaamheden geen tijd. Zie figuur 1.



Figuur 1: Een digitale schakeling.

Ontwerp een programma dat, met gebruik van functies voor elke type poort, het hele schema doorrekent en de resultaten netjes als een waarheidstabel afdrukt.

Bedenk zelf handige namen voor de functies van AND, OR ... en zorg ervoor dat 2-input en 3-input (4-input?) poorten eenvoudig uit elkaar te houden zijn.

Zorg ervoor dat er zo efficiënt mogelijk met het opstellen van de functies wordt omgegaan. Bedenk bijvoorbeeld dat een NAND opgebouwd is uit een AND en een NOT. Het is dus niet nodig een volledige functie voor de NAND te maken, want je kunt gebruik maken van de eerder geschreven functies voor AND en NOT. Alleen de functies voor OR2, AND2 en NOT moeten uitgewerkt worden, de rest van de functies zijn af te leiden uit de eerst genoemde functies.

## 4.8 Maximale

Het bepalen van een maximum uit een reeks getallen wordt vaak gebruikt. Denk hierbij aan het bepalen van de maximale temperatuur van de dag.

Ontwerp een programma dat aan de gebruiker een vijftal gehele getallen vraagt en het maximum van deze getallen afdrukt. Hiervoor moet een aantal functies geschreven en gebruikt worden:

- Als eerste moet er een functie worden geschreven, genaamd `max2`, die het maximum teruggeeft van twee gehele getallen. De getallen moeten als argumenten aan de functie worden meegegeven.
- Vervolgens moet er een functie `leesGetal` worden geschreven die een geheel getal inleest (netjes eerst tekst afdrukken dat een getal moet worden opgegeven), en dit ingelezen getal teruggeeft. Er mogen alleen gehele getallen worden ingelezen: letters en leestekens moeten worden genegeerd.
- Natuurlijk kan er een functie worden geschreven die in één keer het maximum van vijf getallen bepaalt maar dat is niet nodig. Je kan het maximum van drie getallen namelijk óók bepalen met behulp van de `max2`-functie: één van de argumenten is dan het maximum van twee getallen! De `max2`-functie wordt dus gebruikt als argument in een (andere) `max2`-functie. Ontwerp een `max5`-functie door alleen maar `max2`-functies te gebruiken.

Verzin zelf meerdere testen. Zorg dat je deze testen bij de hand hebt tijdens het nakijken.

## 4.9 Gemiddelde

Schrijf een programma dat het gemiddelde van vijf getallen uitrekent.

Er moet een functie `leesGetal` worden geschreven die een geheel getal inleest (netjes eerst tekst afdrukken dat een getal moet worden opgegeven), en dit ingelezen getal teruggeeft. Er mogen alleen getallen worden ingelezen: letters en leestekens moeten worden genegeerd.

Er moet een functie `gemiddelde` worden geschreven die het gemiddelde van de vijf getallen uitrekent en teruggeeft. Uiteraard moeten de vijf ingevoerde getallen als argumenten worden opgegeven.

Het hoofdprogramma moet de bovenstaande functies gebruiken.

Verzin zelf meerdere testen. Zorg dat je deze testen bij de hand hebt tijdens het nakijken.

#### 4.10 Kleinste gemene veelvoud

Er zijn toepassingen waarbij de *kleinste gemene veelvoud* (kgv) van twee positieve getallen bepaald moet worden, bijvoorbeeld bij het gelijknamig maken van twee breuken. Het kgv van twee getallen is het kleinste positieve gehele getal dat een veelvoud is van beide getallen, dus het kleinste positieve gehele getal, waarvan beide getallen deler zijn.

Het kgv van twee getallen  $a$  en  $b$  kan berekend worden met de formule:

$$\text{kgv}(a, b) = \frac{a \cdot b}{\text{ggd}(a, b)} \quad \text{met } a, b > 0$$

waarbij  $\text{ggd}(a, b)$  de *grootste gemene deler* is van  $a$  en  $b$ . Euclides heeft in 300 v.c. een algoritme beschreven (waarschijnlijk niet door hem bedacht) voor het vinden van de ggd. Het is gebaseerd op het recursief aanroepen van een functie:

$$\text{ggd}(a, b) = \begin{cases} a & \text{als } b = 0 \\ \text{ggd}(b, a \bmod b) & \text{als } b > 0 \end{cases}$$

Schrijf een functie `int kgv(int a, int b)` die de kgv van twee getallen bepaalt. Schrijf verder een functie `int ggd(int a, int b)` die de ggd van twee getallen bepaalt door gebruik te maken van de recursieve identiteit.

Schrijf een programma dat de getallen  $a$  en  $b$ , beide geheel een groter dan nul, inleest en de kgv berekent en afdruckt met behulp van de bovenstaande geformuleerde functies voor de kgv en ggd. Zorg ervoor dat bij het inlezen van de getallen voor  $a$  en  $b$  getest wordt of deze getallen groter zijn dan 0.

#### 4.11 Wortel schieten

Heron van Alexandrië heeft heel lang geleden een methode ontwikkeld voor het vinden van de wortel van een positief getal (of nul, da's niet zo moeilijk). Het is gebaseerd op het herhaald doorrekenen van een eenvoudige functiebeschrijving:

$$\begin{aligned} X_0 &= S \\ x_{n+1} &= \frac{1}{2} \left( x_n + \frac{S}{x_n} \right) \\ \sqrt{S} &= \lim_{n \rightarrow \infty} x_n \end{aligned}$$

Hier staat het volgende:

- Als eerste benadering van de wortel ( $x_0$ ) neem je het getal ( $S$ ) waarvan je de wortel wil berekenen.
- Een tweede benadering van de wortel ( $x_1$ ) kan worden gevonden door toepassing van de middelste formule (die met die  $1/2$  erin) door voor  $n$  0 in te vullen ( $n + 1$  wordt dus 1).
- Een nieuwe benadering van de wortel kan worden gevonden door  $n$  steeds met 1 te verhogen en de middelste formule (die met die  $1/2$  erin) herhaald toe te passen. Elke keer als je de middelste formule toepast kom je dichterbij de waarde van de wortel.
- Als je dit oneindig vaak doet, dan heb je de waarde van de wortel exact bepaald. Dat staat in de laatste formule.

Schrijf een programma dat de wortel van een getal groter dan of gelijk aan nul uitrekent. De middelste formule moet als functie worden geïmplementeerd. Er moet een functie `leesGetal` worden geschreven die het getal inleest (netjes eerst tekst afdrukken dat een getal moet worden opgegeven), en dit ingelezen getal teruggeeft. Er mogen alleen getallen worden ingelezen: letters en leestekens moeten worden genegeerd.

Verzin zelf meerdere testen. Zorg dat je deze testen bij de hand hebt tijdens het nakijken.

#### 4.12 Som van de cijfers en modulo 9

Gegeven is de functie `leesCijfer` die een cijfer inleest.

Schrijf een functie `leesEnBerekenSom` die de som van de cijfers van een groot getal bepaalt.

BV. 2348967 -> 2+3+4+8+9+6+7 = 39

Gegeven is dat dit getal zo groot is dat het niet in een integer variabele past. Daarom wordt het getal in de functie `leesEnBerekenSom` cijfer voor cijfer ingelezen met behulp van de functie `leesCijfer`. Als de functie `leesCijfer` geen geldig cijfer in kan lezen dan geeft deze functie de waarde `-1` terug.

Schrijf vervolgens de functie `leesEnBerekenMod9` die een groot getal inleest en de modulo 9 van dit getal berekent. Gegeven is dat dit getal zo groot is dat het niet in een integer variabele past. De modulo 9 van een getal is gelijk aan de modulo 9 van de som van de cijfers van dat getal. Je kunt dus gebruik maken van de functie `leesEnBerekenSom`.

Schrijf ook een hoofdprogramma om de functies `leesEnBerekenSom` en `leesEnBerekenMod9` te testen.

Test het programma met:

---

```
1 Geef (groot getal): 12345678901234567890
2 Som is: 90
3 Mod 9 is: 0
```

---

#### 4.13 Isolated prime

Een priemgetal is een getal dat slechts twee delers heeft: het getal 1 en het getal zelf. Zo is het getal 11 een priemgetal want het is alleen deelbaar door 1 en door 11.

Een *isolated prime* is een priemgetal dat geflankeerd wordt door twee niet-priemgetallen. Dat wil zeggen dat als  $p$  een priemgetal is, dan zijn  $p - 2$  en  $p + 2$  dat niet. Neem het getal 23. Dit is een isolated prime, want 21 en 25 zijn geen priemgetallen.

Schrijf een programma dat een geheel getal vraagt (getal  $> 1$ , want 0 en 1 zijn geen priemgetallen) en afdrukt of het opgegeven getal een isolated prime is. In het programma moet een functie `int isprime(int n)` komen die bepaalt of getal  $n$  een priemgetal is. Daarnaast moet er een functie `int isisolatedprime(int n)` die bepaalt of  $n$  een isolated prime is.

#### 4.14 De Ackermann-functie

In de jaren '20 van de vorige eeuw is een functie bedacht door Wilhelm Ackermann. Het is een recursieve functie van twee variabelen  $m$  en  $n$  (beide  $\geq 0$ ). De functie is gedefinieerd als:

$$A(m, n) = \begin{cases} n + 1 & \text{if } m = 0 \\ A(m - 1, 1) & \text{if } m > 0 \text{ and } n = 0 \\ A(m - 1, A(m, n - 1)) & \text{if } m > 0 \text{ and } n > 0. \end{cases}$$

Schrijf een programma dat de getallen  $m$  en  $n$ , beide geheel een groter dan of gelijk aan nul, inleest en met behulp van een functie de Ackermann-functie berekent en afdrukt met behulp van de bovenstaande recursieve identiteit.

#### 4.15 The sinc-functie

In analoge en digitale filters komt veelvuldig de sinc-functie voor. De functie is gedefinieerd als:

$$\text{sinc } x = \frac{\sin x}{x}$$

Schrijf een functie `double sinc(double x)` die sinc-waarde bepaalt van het getal `x`. Druk de waarden af van `x` en `sinc(x)` van 0,0 t/m 6,0 met een stapgrootte van 0,1.

#### 4.16 Aantal delers van een geheel getal

In de getaltheorie komt een bepaalde klasse van getallen voor met de eigenschap dat zo'n getal alleen deelbaar is 1 en zichzelf. We noemen zo'n getal een *priemgetal*. Priemgetallen spelen een belangrijke rol bij encryptie.

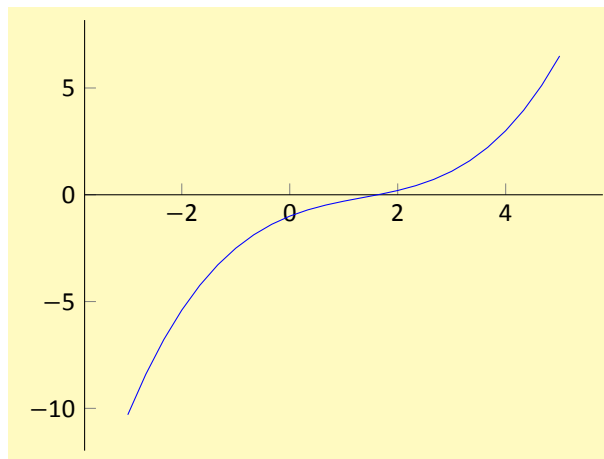
Schrijf een functie `int aantal_delers(int getal)` die van het opgegeven getal het aantal delers bepaalt. Merk op dat 1 ook een deler is. Het getal moet groter zijn dan 1. Schrijf een hoofdprogramma dat de gebruiker vraagt een getal in te voeren (netjes een melding geven wat er gevraagd wordt en testen of het getal groter is dan 1) en het aantal delers afdruckt m.b.v. de functie `aantal_delers`.

#### 4.17 Nulpunten van een functie bepalen

Gegeven is de functie:

$$f(x) = \frac{1}{10}x^3 - \frac{2}{5}x^2 + x - 1 \quad (1)$$

De grafiek van de functie is te zien in de onderstaande figuur. We zien dat het *nulpunt*, waarbij de y-coördinaat 0 is, tussen  $x = 0$  en  $x = 2$  ligt. Met behulp van de *bisectie-methode* (ook wel halveringsmethode genoemd) kan de x-coördinaat van het nulpunt met willekeurige precisie benaderd worden. Zie voor een uitleg [https://en.wikipedia.org/wiki/Bisection\\_method](https://en.wikipedia.org/wiki/Bisection_method).



Schrijf een programma voor het vinden van de x-coördinaat. Maak gebruik van een functie om de y-coördinaat te berekenen bij een gegeven x-coördinaat. Maak gebruik van een functie die het nulpunt willekeurig dicht benaderd. De functie `main` is dus zeer compact en roept alleen functies aan.

## 5 Arrays

### 5.1 Inleiding

In de les heb je het een en ander geleerd over arrays. Een array is een lijst van variabelen van het zelfde type. Je kan een array indexeren. Dat houdt in dat je een *element* uit de lijst kan gebruiken in je bewerkingen. Hiervoor gebruik je blokhaken. Een korte introductie over arrays is te vinden in het boek.

Hieronder is een programma dat een aantal waarden inleest in een array en vervolgens het gemiddelde van deze waarden afdrukt (het is in dit geval eigenlijk niet nodig om de getallen in een array op te slaan). Maak een project aan en plaats daarin de onderstaande code. Laat zien dat het werkt.

```
1 #include <stdio.h>
2
3 /* For Visual Studio */
4 #pragma warning(disable : 4996)
5
6 #define AANTAL 10
7
8 int main(void) {
9     double temperatuur[AANTAL];
10    int i;
11    double temp_acc;
12
13    for (i = 0; i < AANTAL; i = i + 1) {
14        do {
15            printf("Geef temperatuur %d op: ", i + 1);
16            } while (scanf("%lf", &temperatuur[i]) != 1);
17    }
18
19    temp_acc = 0.0;
20    for (i = 0; i < AANTAL; i = i + 1) {
21        temp_acc = temp_acc + temperatuur[i];
22    }
23
24    printf("De gemiddelde temperatuur is %.2f\n\n", temp_acc / AANTAL);
25
26    printf("Druk op enter om het programma af te sluiten.");
27    getchar();
28    return 0;
29 }
```

In het eerste deel van het programma worden de getallen ingelezen (dit mogen floating point-getallen zijn). In het tweede deel wordt het gemiddelde berekend van alle ingevoerde getallen. Het is eenvoudig om het aantal getallen aan te passen. We moeten dan de *macro* AANTAL wijzigen. Om twintig getallen in te lezen, gebruiken we de regel:

```
1 #define AANTAL 20
```



Let op! Je moet twee van de onderstaande opdrachten maken. De practicumdocent bepaalt welke opdrachten je moet maken.

## 5.2 Omkeren

C kent geen primitief type voor een string (primitieve types zijn o.a. `int` en `double`). Een string met karakters in C is niets anders dan een array van `char`'s, afgesloten met een nul-karakter (`'\0'`). Er zijn veel functies beschikbaar voor het manipuleren van strings, zoals het bepalen van de lengte of het kopiëren van strings. Informatie is te vinden op <https://en.wikipedia.org/wiki/String.h>.

Schrijf een programma dat een string inleest van het toetsenbord en deze string achterstevoren afdruckt. Hiervoor moet een functie `reverse` worden geschreven met als parameter de string. Deze functie zorgt er voor dat de string achterstevoren wordt gezet. Het afdrucken gebeurt in het hoofdprogramma. Het gebruik van de functie `strrev` is niet toegestaan.

Verzin zelf testdata. Zorg dat je deze testdata bij de hand hebt tijdens het nakijken.

## 5.3 Voortschrijden

Het bijhouden van de temperatuur wordt al jaren gedaan. Meestal wordt hierbij dan een gemiddelde temperatuur berekend. Dit is eenvoudig, zie opgave 5.1. Maar er is nog een soort gemiddelde: het voortschrijdend gemiddelde. Dit wordt bijvoorbeeld gebruikt bij beurskoersen<sup>5</sup>.

Schrijf een programma dat steeds het voortschrijdend gemiddelde bepaalt van de laatste 10 koersstanden. Na elke berekening moet een nieuwe koerswaarde ( $> 0$ ) worden ingelezen. Gebruik een array om de koersstanden op te slaan.

Verzin zelf testdata. Zorg dat je deze testdata bij de hand hebt tijdens het nakijken.

## 5.4 Vergelijken

Schrijf een programma dat twee arrays van integers van gelijke lengte vergelijkt. De arrays zijn identiek (gelijk) als elk element van de eerste array overeenkomt met het element op dezelfde positie in de tweede array.

---

```
1 int rij1[] = { 1, 7, 5, 3};  
2 int rij2[] = { 1, 7, 5, 3};
```

---

zijn identiek omdat de lengte gelijk is en omdat elk element overeenkomt.

## 5.5 Histogram

Schrijf een programma dat de gehele getallen 0 t/m 10 inleest en bijhoudt hoe vaak elk getal is ingevoerd. Getallen kleiner dan  $-1$  of groter dan 10 moeten worden genegeerd. Het getal  $-1$  sluit de invoerprocedure af. Daarna drukt het programma de aantallen af in de vorm van een histogram<sup>6</sup>. Dit kan bijvoorbeeld elegant met (een rij van) sterretjes. Let erop dat de uitvoer geschaald moet worden; het getal dat het meest is ingevoerd moet met 30 sterretjes worden afgedrukt. Het afdrucken mag horizontaal of verticaal.

Verzin zelf testdata. Zorg dat je deze testdata bij de hand hebt tijdens het nakijken.

## 5.6 Sorteren met “selection sort”

Schrijf een programma dat tien gehele getallen in een array inleest en deze getallen oplopend sorteert. Dat kan als volgt: zoek het minimum van alle getallen in de array en verwissel dit getal met het eerste getal in de array. Zoek vervolgens het minimum van de getallen met index 1 t/m 9 in de array en verwissel dit getal met het getal met de index 1 (het tweede getal) in de array. Zoek vervolgens het minimum van de getallen met index 2 t/m 9 in de array en verwissel dit getal met het getal met de index 2 in de array. Dit moet je herhalen totdat de gehele array gesorteerd is.

---

<sup>5</sup> Zie [https://nl.wikipedia.org/wiki/Voortschrijdend\\_gemiddelde](https://nl.wikipedia.org/wiki/Voortschrijdend_gemiddelde)

<sup>6</sup> Zie <https://en.wikipedia.org/wiki/Histogram>

NB: Dit algoritme wordt *selection sort* genoemd, het is een leuke programmeeroefening maar een erg inefficiënte sorteermethode.

Verzin zelf testdata. Zorg dat je deze testdata bij de hand hebt tijdens het nakijken.

## 5.7 Palindroom

C kent geen primitief type voor een string (primitieve types zijn o.a. `int` en `double`). Een string met karakters is in C niets anders dan een array van `char`'s, afgesloten met een nul-karakter (`'\0'`). Er zijn veel functies beschikbaar voor het manipuleren van strings, zoals het bepalen van de lengte of het kopiëren van strings. Informatie is te vinden op <https://en.wikipedia.org/wiki/String.h>.

Schrijf een programma dat een string inleest van het toetsenbord en bepaalt of deze string een palindroom<sup>7</sup> is. Hiervoor moet een functie `is_palindroom` worden geschreven met als argument de string. De functie geeft een 1 terug als de string een palindroom is, anders geeft de functie een 0 terug. Het afdrucken gebeurt in het hoofdprogramma. Het gebruik van de functie `strrev` is niet toegestaan.

Verzin zelf testdata. Zorg dat je deze testdata bij de hand hebt tijdens het nakijken.

## 5.8 Sorteren met “bubble sort”

Schrijf een programma dat tien gehele getallen in een array inleest en deze getallen oplopend sorteert. Dat kan als volgt: als het eerste getal in de array groter is dan het tweede getal, dan verwissel je die twee. Dit kan dan ook voor het tweede en derde getal etc. dus voor de hele array. Dit moet je herhalen totdat er geen verwisselingen meer nodig zijn, dan is de array gesorteerd.

NB: Dit algoritme wordt *bubble sort* genoemd, het is een leuke programmeeroefening maar een erg inefficiënte sorteermethode.

Verzin zelf testdata. Zorg dat je deze testdata bij de hand hebt tijdens het nakijken.

## 5.9 Zoeken

Schrijf een programma dat een string inleest en bepaalt of dit woord in een gesorteerde lijst met 127 woorden voorkomt. Hiervoor moet je het ingevoerde woord vergelijken met de woorden in de lijst. Merk op dat je met maximaal zeven vergelijk-opdrachten kan bepalen of het ingevoerde woord in de gesorteerde lijst voorkomt of niet. Als je het woord wat je zoekt namelijk vergelijkt met het woord met index 63 in de lijst dan kun je het meteen vinden (als het woord dat je zoekt gelijk is aan het woord dat op index 63 staat) of weet je of het woord gezocht moet worden in de lijst van index 0 t/m 62 (als het woord dat je zoekt kleiner is dan het woord dat op index 63 staat) of dat het woord gezocht moet worden in de lijst van index 64 t/m 126 (als het woord dat je zoekt groter is dan het woord dat op index 63 staat). Op deze manier halveer je steeds het deel van de lijst dat je nog moet doorzoeken. Na 7 keer halveren heb je de gehele lijst van  $127 = 2^7 - 1$  woorden doorzocht. Je programma mag dus niet meer dan zeven vergelijk-opdrachten uitvoeren. Een programma waarin een lijst met 127 gesorteerde meisjesnamen staat kun je vinden op BlackBoard.

Verzin zelf testdata. Zorg dat je deze testdata bij de hand hebt tijdens het nakijken.

## 5.10 Sinustabel

Het uitrekenen van een sinus van een hoek kost veel tijd. Slimmer is om een tabel aan te leggen van sinuswaarden bij verschillende hoeken. Nadeel is dat niet van elke willekeurige hoek de sinus kan worden opgevraagd.

Schrijf een programma dat een array vult met de sinuswaarden van 0 tot 90 in hele graden. Daarna vraagt het programma om een geheel getal tussen 0 en 360 en drukt de bijbehorende sinuswaarde met behulp van de tabel af. Dit doet het programma totdat `-1` wordt ingevoerd.

---

<sup>7</sup> Zie <http://nl.wikipedia.org/wiki/Palindroom>

Noot: deze oplossing wordt ook gebruikt bij het ontwerpen van digitale systemen. Het opzoeken van waarden is vele malen sneller dan het uitrekenen ervan.

### 5.11 Cesuur aanpassen

Bij sommige opleidingen wordt de cesuur van een toets aangepast zodat het percentage voldoende op of boven 70% ligt. Als blijkt dat een toets slecht is gemaakt dan moeten er punten worden bijgeteld zodanig dat het percentage voldoende op of boven 70% ligt. Is de toets goed gemaakt, dan moeten er geen punten worden bijgeteld.

Schrijf een programma dat een aantal resultaten (tussen 0,0 en 10,0) inleest. Dit aantal moet door de gebruiker worden opgegeven. Het minimum aantal is 10 en het maximum aantal is 100. Daarna moet bepaald worden of het aantal voldoende onder of boven de 70% ligt. Als het percentage gelijk is aan of groter is dan 70%, dan hoeft er geen bijtelling plaats te vinden. Als het percentage te laag is, dan moet er wel bijtelling plaats vinden. Dit kan door steeds 0,1 bij de resultaten op te tellen en daarna te bepalen of de 70%-grens al bereikt is. Let erop dat resultaten nooit hoger kunnen zijn dan 10,0.

Verzin zelf testdata. Zorg dat je deze testdata bij de hand hebt tijdens het nakijken.

### 5.12 Lineaire regressie

In de statistiek wordt gebruikt gemaakt van lineaire regressie. Stel, je meet de waarden van spanning en stroom bij een bepaalde weerstand. In principe is dit een perfecte rechte lijn aangezien de wet van Ohm een lineair verband weergeeft tussen spanning en stroom. Door meetfouten en onnauwkeurigheden in de schakeling liggen de meetgegevens niet op een perfecte lijn, ze liggen “rond” of “in de buurt” van die lijn. Met behulp van statistiek kan je de rechte lijn vinden die het best past bij de meetgegevens.

De gebruiker moet een aantal x-y paren invoeren. Dit aantal wordt gevraagd aan de gebruiker en ligt tussen 2 en 100. Daarna moeten de x-y paren worden gevraagd. Vervolgens berekent het programma de rechte lijn die het best past bij de meetgegevens.

Deze rechte lijn wordt gegeven door  $y = \beta \cdot x + \alpha$ . Om de parameters  $\beta$  en  $\alpha$  te vinden, moet je de volgende berekeningen doen:

$$\beta = \frac{\sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})}{\sum_{i=1}^n (x_i - \bar{x})^2}$$
$$\alpha = \bar{y} - \beta \bar{x}$$

Hierin zijn  $\bar{x}$  en  $\bar{y}$  de rekenkundige gemiddelden van de x-gegevens resp. de y-gegevens.

Verzin zelf testdata. Zorg dat je deze testdata bij de hand hebt tijdens het nakijken.

### 5.13 Controle identificatiecode van Nederland eurobankbiljet

Schrijf een functie `int controleerNedEuroID(char id[])` die de identificatiecode van een Nederlands eurobankbiljet controleert. De parameter `id` is een C-string (een array van karakters afgesloten met een nul-karakter). De identificatiecode van een Nederlands eurobankbiljet bestaat uit de letter P gevolgd door 11 cijfers. De som van deze 11 cijfers modulo 9 moet 1 zijn. Zie eventueel: [https://nl.wikipedia.org/wiki/Eurobankbiljetten#Nationale\\_identificatiecodes](https://nl.wikipedia.org/wiki/Eurobankbiljetten#Nationale_identificatiecodes). Als de ingevoerde identificatiecode geldig is dan moet de functie `controleerNedEuroID` de waarde 1 teruggeven en anders moet deze functie de waarde 0 teruggeven.

Schrijf een hoofdprogramma om de functie `controleerNedEuroID` grondig te testen.

Test het programma met:

---

```
1 int res = controleerNedEuroID("P12345678901");
```

---

Na het uitvoeren moet de variabele `res` de waarde 0 hebben.

Bepaal zelf ook een aantal testwaardes.

## 5.14 Doorsnede

Schrijf een functie `int doorsnede(int a[], int aantal_a, int b[], int aantal_b)` die de *doorsnede*<sup>8</sup> van de array's `a` en `b` in de array `a` plaatst. De parameter `aantal_a` bevat het aantal elementen in de array `a` en de parameter `aantal_b` bevat het aantal elementen in de array `b`. Je mag ervan uit gaan dat een integer hooguit eenmaal in elke array kan voorkomen. De functie moet het aantal getallen in de doorsnede teruggeven als return waarde.

Test de functie met het onderstaande hoofdprogramma:

---

```
1 int rij1[] = { 23, 3, 56, 6, 5 };
2 int rij2[] = { 12, 6, 3, 18, 7, 10, 56, 11 };
3 int aantal_doorsnede = doorsnede(rij1, sizeof rij1 / sizeof rij1[0], rij2,
  ↳ sizeof rij2 / sizeof rij2[0]);
4 int i;
5
6 for (i = 0; i < aantal_doorsnede; i = i + 1) {
7     printf("%d ", rij1[i]);
8 }
```

---

moet de volgende uitvoer geven: 3 6 56

Bepaal zelf ook een aantal testwaardes.

## 5.15 Uniek

Schrijf een functie `int uniek(int a[], int aantal_a)` die alle getallen die meer dan één keer in de array `a` voorkomen verwijderd uit deze array. De parameter `aantal_a` bevat het aantal elementen in de array `a`. De functie moet het aantal unieke getallen in array `a` teruggeven als return waarde.

Schrijf ook een hoofdprogramma om de functie `uniek` grondig te testen.

Test het programma met:

De code:

---

```
1 int rij[] = { 23, 3, 56, 3, 23, 5, 12, 6, 3, 18, 7, 10, 56, 11 };
2 int aantal_uniek = uniek(rij, sizeof rij / sizeof rij[0]);
3 int i;
4
5 for (i = 0; i < aantal_uniek; i = i + 1) {
6     printf("%d ", rij[i]);
7 }
```

---

kan de volgende uitvoer geven: 5 12 6 18 7 10 11.

Bereken zelf ook een aantal testwaardes.

---

<sup>8</sup> Zie [https://nl.wikipedia.org/wiki/Doorsnede\\_\(verzamelingenleer\)](https://nl.wikipedia.org/wiki/Doorsnede_(verzamelingenleer))

## 5.16 Run Length encoding

Run Length Encoding (RLE) is een van de oudste compressiemethoden. Een reeks identieke, naast elkaar geplaatste karakters (een Run) wordt vervangen door het aantal karakters (Length) en het karakter.

Zo wordt de invoer van de string:

```
1 aaaabbbbbccccccdaaaa
```

gecodeerd als:

```
1 4a6b8c1d4a
```

Merk op dat de uitvoer groter wordt als de karakters steeds afwisselen. Zo wordt de string `abcd` gecodeerd als `1a1b1c1d`.

Schrijf een programma dat een string inleest en deze string RLE-gecodeerd in een andere string plaatst. Gebruik hiervoor de functie:

```
1 double rlecompress(char invoerstring[], char rlestring[]).
```

Zorg ervoor dat de string `rlestring` groot genoeg is om de gecomprimeerde string op te slaan. De return-waarde is de compressieratio: de lengte van `rlestring` gedeeld door de lengte van `invoerstring`.

Verzin zelf testdata. Zorg dat je deze testdata bij de hand hebt tijdens het nakijken.

## 5.17 Rolling dice

Het gooien van dobbelstenen doen we al sinds mensenheugenis. Er zijn veel spelletjes waarbij twee dobbelstenen in één worp worden gegooid. Maar hoe vaak wordt nu 7 als het aantal ogen gegooid? En hoe vaak wordt nu 2 als het aantal ogen gegooid? Dat willen we uitzoeken.

Schrijf een programma dat de gebruiker vraagt hoe vaak de twee dobbelstenen gegooid moeten worden (`getal > 2`). Maak een functie `roll_dice()` die het gooien van één dobbelsteen simuleert. Met deze functie kan je het aantal gegooide ogen van twee dobbelstenen simuleren. Houd in een array bij hoe vaak een bepaald aantal ogen wordt gegooid. Druk aan het einde van de simulatie van alle mogelijke combinaties het gegooide aantal ogen af.

Bij deze opdracht moet je gebruik maken van de *pseudo random generator* in C. De term “pseudo” geeft aan dat het hier om een random generator gaat die niet geheel random is (maar goed genoeg voor ons experiment). Om te beginnen moet je een *seed* zetten. Dat is het startpunt van onze reeks. Daarna kan je via de functie `rand()` een random getal opvragen. Zie voor meer informatie: [http://www.gnu.org/software/libc/manual/html\\_node/ISO-Random.html#ISO-Random](http://www.gnu.org/software/libc/manual/html_node/ISO-Random.html#ISO-Random).

## 5.18 Overtollige spaties en tabs verwijderen

Bij het *parsen* (ontleden) van een string is het soms nodig om overtollige spaties en tabs te verwijderen. Met het verwijderen wordt bedoeld dat een reeks van aaneengesloten spaties en/of tabs worden vervangen door één spatie. Spaties aan het begin en einde van een string moeten in het geheel verwijderd worden. Dit wordt *trimming* genoemd. Hieronder zie je een voorbeeld waarbij spaties zijn vervangen door het `_`-symbool en tabs zijn vervangen door het `__`-symbool.

```
1 ___Hallo, __dit____is____een__string.____
```

Deze string wordt omgezet in

```
1 Hallo,_dit_is_een_string.
```

Het verwijderen moet in de ingevoerde string gebeuren, er mag geen nieuwe string aangemaakt worden. Ga ervan uit dat de ingevoerde string niet langer is dan 100 karakters.

## 5.19 Waar of niet waar

Gegeven is de volgende bewering:

Deze zin bevat 1 keer een 0, 1 keer een 1, 1 keer een 2, 1 keer een 3, 1 keer een 4, 1 keer een 5, 1 keer een 6, 1 keer een 7, 1 keer een 8 en 1 keer een 9.

We bekijken **alleen** de cijfers en getallen. Natuurlijk is de bewering niet waar, er zijn immers 11 enen. Toch is er een zin mogelijk waarbij het aantal cijfers en de getallen kloppen. Schrijf een programma dat vanuit een beginsituatie (stel alle aantallen 1) iteratief naar een mogelijke oplossing zoekt.

Hint: maak gebruik van twee arrays. Array `int a[10]` bevat het aantal cijfers in getalvorm, dus element `a[0]` bevat het aantal nullen in getalvorm. In array `int b[10]` houd je bij hoe vaak een cijfer voorkomt in de getallen van array `a`. Dus `b[1]` zou “10” kunnen bevatten omdat er in de getallen van array `a` 10 keer een 1 voorkomt. Ga ervan uit dat een cijfer niet meer dan 99 keer voorkomt. Bedenk ook dat het aantal voorkomingen van een cijfer minstens 1 is (kijk hierbij naar de zin).

## 5.20 Queue

Een *queue* (Nederlands: rij of *wachtrij*) is een datastructuur met de volgende kenmerken: de queue heeft een vaste lengte. Er zijn drie functies: initialiseer de queue, zet iets in de queue en haal iets van de queue. De functie `init` initialiseert de queue; de queue wordt dan leeggemaakt. De functie `put` zet iets in de queue zonder de overige informatie aan te passen. De functie `get` haalt iets uit de queue, namelijk de informatie die als eerste in de queue is geplaatst. Daarna is die informatie verdwenen en komt eventueel de dan opvolgende informatie beschikbaar. Zodoende “schuift” de informatie van `put` naar `get`.

Een mogelijke sequentie is hieronder te zien:

---

```
1 init();
2 put(43);
3 put(56);
4 put(17);
5 int i = get(); // i == 43
6 int j = get(); // j == 56
7 int k = get(); // k == 17
```

---

Let erop dat de queue vol kan zijn, dus een aanroep met `put` moet dan een foutmelding geven. Let erop dat de queue leeg kan zijn, dus een aanroep met `get` moet dan een foutmelding geven. Ontwerp de queue en de functies van de queue voor integers.

## 6 Structures

### 6.1 Inleiding

Een structure is een handige manier om bij elkaar behorende gegevens te gebruiken onder één gemeenschappelijke noemer. Het voordeel van een structure is dat het als geheel kan worden meegegeven als een argument in een functie en dat een functie een complete structure kan retourneren. Daarnaast is het makkelijk om een array van structures te maken zodat je dataverwerking kan toepassen

Als voorbeeld zie je hier een structure die toetsresultaten kan opslaan. Een toetsresultaat is gekoppeld aan een student. Van de student wordt het studentnummer en de achternaam, samen met het resultaat in een structure opgeslagen.

```
1 struct student_struct {
2     int idcode;           // An integer
3     char achternaam[30];  // A string of 29 characters
4     char klas[10];        // A string of 9 characters
5     double resultaat;     // A double between 0.0 and 10.0
6 };
7
8 struct student_struct jan;
9
10 jan.idcode = 21011009;    // an integer
11 strcpy(jan.achternaam, "Klaassen"); // no more than 29 characters!
12 strcpy(jan.klas, "ep13"); // no more than 9 characters!
13 jan.resultaat = 7.3;      // a double between 0.0 and 10.0
14
15 printf("%8d %-19s %-9s %4.1f\n", jan.idcode, jan.achternaam, jan.klas,
    ↪ jan.resultaat);
```

Je kan de *members* van de structure (idcode, achternaam, resultaat) gewoon gebruiken als andere variabelen. Dit is te zien in de toekenningen en in de `printf`-regel.

Het is ook mogelijk om een *array van structures* te definiëren, bijvoorbeeld alle studenten in een klas. We gebruiken hiervoor weer `struct student_struct` maar geven nu een array op. Met behulp van een `for`-lus kan je langs alle elementen uit de array “lopen”.

```
1 struct student_struct klas[50];
2
3 ... /* Hier moeten de gegevens worden ingevuld */
4
5 /* Afdrukken van de gegevens */
6 for (int i = 0; i < 50; i++) {
7     printf("%d %s %f\n", klas[i].idcode,
8                 klas[i].achternaam,
9                 klas[i].resultaat);
10 }
```



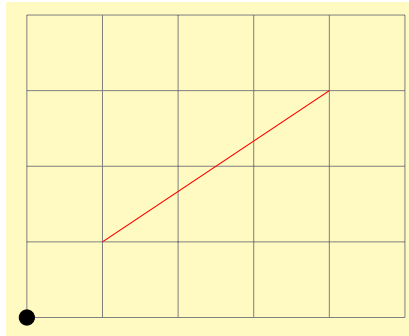
Let op! Je moet twee van de onderstaande opdrachten maken. De practicumdocent bepaalt welke opdrachten je moet maken.

## 6.2 Afstand tussen twee punten

Gegeven zijn twee punten in een 2-dimensionaal vlak. De punten worden gegeven via de structure:

```
1 struct punt {  
2     double x, y;  
3 }
```

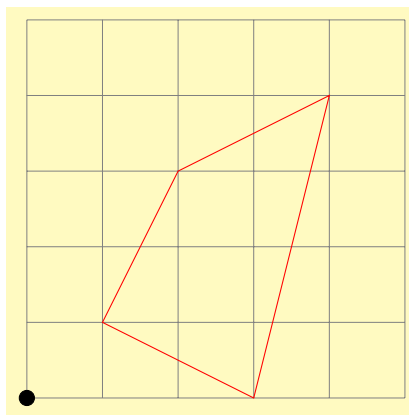
In de onderstaande figuur is een uitbeelding te zien van de twee punten en een lijn tussen de twee punten.



Schrijf een programma dat de twee structures inleest en vervolgens de afstand tussen deze twee punten berekent en afdrukt. Hint: gebruik de stelling van Pythagoras.

## 6.3 Omtrek van een polygon

Een *polygon* is een 2-dimensionale figuur waarbij een aantal punten verbonden worden met lijnen. Een voorbeeld van een polygon is te zien in onderstaande figuur. Schrijf een programma dat de punten van de polygon inleest en de omtrek van de polygon berekent. Ga ervan uit dat maximaal 5 punten worden ingelezen. Maak gebruik van de structure in opdracht 6.2. Hint: gebruik de stelling van Pythagoras.



## 6.4 Tijd naar seconden

Gegeven is de variabele `tijd` met de volgende definitie:

```
1 struct urenminutenseconden {  
2     int uren;  
3     int minuten;  
4     int seconden;  
5 } tijd;
```

Schrijf een functie `lees_tijd` die de uren, minuten en seconden inleest in de structuur `tijd`. Schrijf vervolgens een functie `bereken_seconden` die de structuur meekrijgt als parameter en het aantal seconden die deze tijd vertegenwoordigt, teruggeeft. Schrijf een hoofdprogramma die de functies test.

## 6.5 Seconden naar tijd

Schrijf een programma die het aantal seconden inleest en omzet naar uren, minuten en seconden. Dit omzetten moet gedaan worden in een functie. De functie krijgt het aantal seconden mee en geeft de uren, minuten en seconden in een structuur terug. Deze structuur is te vinden in opdracht 6.4. Schrijf een hoofdprogramma om de functie te testen.

## 6.6 Tijdsverschil

Schrijf een programma dat twee tijden inleest (in uren, minuten en seconden) en het verschil tussen deze tijden (in seconden) afdrukt. Gebruik de structuur uit opdracht 6.4.

## 6.7 Studenten

In de inleiding (paragraaf 6.1) wordt de structuur `student_struct` beschreven. Hiermee kunnen we gegevens van studenten en het resultaat van een toets opslaan. Gebruik deze structuur.

Schrijf een programma dat de gegevens van 5 van deze studenten inleest en afdrukt. Gebruik hiervoor een functie `lees_student` en `print_student`. Gebruik Call by Value voor argument/parameter-overdracht en returnwaarde. Bij het afdrukken moeten de gegevens netjes onder elkaar worden afgedrukt, alsof het in kolommen is afgedrukt.

## 6.8 Studenten again

Als opdracht 6.7 maar realiseer nu ook een functie `maximum` die het maximum behaalde resultaat retourneert. Gebruik de functie in een array van structures van studenten.

## 7 Pointers

### 7.1 Introductie

In de les heb je het een en ander geleerd over pointers. Een pointer is een variabele die als inhoud een *adres* bevat. Dit adres is dan het adres waar een andere variabele ligt opgeslagen. Zodoende “wijst” de pointer dus naar de variabele. In het boek kan je meer vinden over pointers.

Hieronder is een programma gegeven dat wat meer vertelt over pointers. Kopieer het programma in een nieuw project en start het programma. Bestudeer de uitvoer nauwgezet.

---

```
1 #include <stdio.h>
2
3 int main(void) {
4
5     int i = 2;
6     int *pi;
7     int ai[10] = { 1, 2, 3, 0 };
8
9     pi = &i;
10
11     printf("Adres van i: %p\n", &i);
12     printf("Inhoud van pi: %p\n", pi);
13
14     printf("Inhoud van i: %d\n", i);
15     printf("Via pointer pi: %d\n\n", *pi);
16
17     printf("Beginadres array ai: %p\n", &ai[0]);
18     printf("Beginadres array ai: %p\n", ai);
19
20     printf("Inhoud van ai[1]: %d\n", ai[1]);
21     printf("Inhoud van ai[1]: %d\n\n", *(ai+1));
22
23     printf("Startadres van functie main: %p\n", main);
24     printf("Startadres van functie printf: %p\n", printf);
25
26     return 0;
27 }
```

---

Een pointer wordt in C aangegeven door een \* voor de variabelenaam te plaatsen. Bij het definitie van een variabele, zoals in `int *pi`; zal de compiler een pointer aanmaken. Bij gebruik in een statement, zoals in `i = *pi`; wordt de inhoud van de variabele waar de pointer naar toewijst gebruikt.

Het adres van een variabele is op te vragen door een & voor de variabele te plaatsen. Dus het statement `pi = &i`; zorgt ervoor dat pointer pi “wijst” naar variabele i.

Arrays kunnen in C via pointers bewerkt worden. Sterker nog, er is geen fundamenteel verschil tussen arrays en pointers, alleen de notatie is anders. De naam van de array mag bijvoorbeeld als pointer naar het eerste element uit de array gebruikt worden. Om het tweede element van een array te gebruiken kunnen we dus `i = ai[1]`; óf `i = *(ai+1)`; . De haakjes zijn nodig omdat \* een hogere prioriteit heeft dan +.

Ook de naam van een functie, bijvoorbeeld `printf`, is een pointer, maar nu naar het begin van een functie. Dit worden functiepointers genoemd. Dit soort pointers komt niet zoveel voor.



Let op! Je moet twee van de onderstaande opdrachten maken. De practicumdocent bepaalt welke opdrachten je moet maken.

## 7.2 Array afdrukken

Schrijf een functie die een array van tien integers met behulp van een pointer afdruckt. Definieer zelf de inhoud van de array. Test de functie met een hoofdprogramma.

## 7.3 Array afdrukken, reprise

Schrijf een functie die een array van *variabele* lengte gevuld met integers met behulp van een pointer afdruckt. De lengte moet dus tijdens run-time berekend en meegegeven worden. Definieer zelf de inhoud van de array. Test de functie met een hoofdprogramma.

## 7.4 String achterstevoren afdrukken

Schrijf een functie die een meegegeven string (dus als argument) van willekeurige lengte achterstevoren afdruckt. Maak gebruik van een pointer. De string mag niet aangepast worden in de functie. Test de functie met een hoofdprogramma.

## 7.5 Woorden tellen

Schrijf een functie die in een meegegeven string (dus als argument) van willekeurige lengte het aantal woorden telt. De woorden in de string worden gescheiden door spaties. Vergeet niet de woorden aan het begin en einde van de string mee te tellen. Het getelde aantal woorden wordt als returnwaarde teruggegeven. Maak gebruik van pointers. Test de functie met een hoofdprogramma.

## 7.6 Array van integers omkeren

Schrijf een functie die een meegegeven array van integers en een meegegeven lengte van de array in elementen, omkeert. Het eerste en laatste element worden dus verwisseld, het tweede en het een-na-laatste element worden verwisseld, enzovoorts. Zo ook met de andere elementen. Maak gebruik van pointers. Test de functie met een hoofdprogramma.

## 7.7 Zoek het grootste getal

Gegeven is een array van integers (die moet je zelf vullen). Schrijf een functie die van de meegegeven array het grootste getal bepaalt. Natuurlijk moet ook de lengte van de array meegegeven worden. Het *adres* van het grootste getal moet via een pointer worden teruggegeven. Schrijf een hoofdprogramma die de functie aanroept en via de teruggegeven pointer het grootste getal afdruckt.

## 7.8 Op volgorde

Schrijf een functie `opvolgorde` die drie pointers naar integers meekrijgt en via deze pointers de integers op oplopende volgorde sorteert. Stel dat de integers *i*, *j* en *k* zijn. Dan moet na de aanroep

---

```
1 opvolgorde(&i, &j, &k)
```

---

gelden dat  $i \leq j \leq k$ .

## 7.9 Zoek laatste getal

Schrijf een functie die in een meegegeven array van integers zoekt naar een eveneens meegegeven getal. De lengte van de array moet natuurlijk ook meegegeven worden. De *laatste* voorkoming van dit getal moet als adres worden teruggegeven. Wordt het getal niet gevonden, dan moet de NULL-pointer worden teruggegeven. Schrijf een hoofdprogramma dat je functie test.

## 7.10 Vervangen van karakter

Schrijf een functie die in een meegegeven string, het meegegeven karakter vervangt door een tweede meegegeven karakter. Maak gebruik van pointers in deze functie. De functie geeft het aantal vervangingen terug als returnwaarde (en dat kan ook 0 zijn). Test de functie met een hoofdprogramma.

## 7.11 Vervangen van woord

Voor het vervangen van een woord in een string is in C geen functie beschikbaar. Die zal je dus zelf moeten schrijven. Schrijf een functie:

---

```
1 int stringreplace(char *str, char *what, char *with)
```

---

die in de string *str* zoekt naar het woord aangegeven door *what* en vervangt door het woord aangegeven door *with*. De return-waarde is het aantal vervangingen en kan natuurlijk ook 0 zijn. Schrijf een hoofdprogramma om de functie te testen. Noot: let erop dat de lengte van *with* groter kan zijn dan de lengte van *what*. Zorg ervoor dat de ruimte van de string *str* groot genoeg is. Maak gebruik van pointers.

## 7.12 Dynamische array

Schrijf een programma dat met behulp van *dynamische geheugenallocatie* een array van integers van willekeurige lengte van het toetsenbord inleest en deze array netjes afdrukt. Het aantal integers moet als getal van het toetsenbord worden ingelezen. De spil in deze opdracht is het gebruik van de functie `calloc`. Zie bijvoorbeeld <https://www.cplusplus.com/reference/cstdlib/calloc/>.

## 7.13 Dynamische array omkeren

Schrijf een programma dat met behulp van *dynamische geheugenallocatie* een array van integers van willekeurige lengte van het toetsenbord inleest en deze array *omgekeerd* afdrukt. Het aantal integers moet als getal van het toetsenbord worden ingelezen. De spil in deze opdracht is het gebruik van de functie `calloc`. Zie bijvoorbeeld <https://www.cplusplus.com/reference/cstdlib/calloc/>.

## 7.14 Dynamisch inlezen van een string

Schrijf een programma dat met behulp van *dynamische geheugenallocatie* een string inleest. Eerst moet worden opgegeven hoe lang de string is. Vervolgens moet de string van het toetsenbord worden ingelezen. Overtollige karakters moeten worden genegeerd.

## 7.15 Dynamisch inlezen van een $n \times n$ -matrix

Schrijf een programma dat met behulp van *dynamische geheugenallocatie* een  $n \times n$  matrix inleest. De grootte van  $n$  moet worden ingelezen van het toetsenbord. Daarna moeten de elementen van de matrix worden ingelezen. Als alle elementen zijn ingelezen, moet de matrix op het beeldscherm worden afgedrukt. Gebruik pointers voor het vullen en afdrukken. De spil in deze opdracht is het gebruik van de functie `calloc`. Zie bijvoorbeeld <https://www.cplusplus.com/reference/cstdlib/calloc/>.

## 7.16 Onbeperkt

Het inlezen van een regel tekst van het toetsenbord is niet echt lastig. Het wordt pas problematisch als er meer tekens worden ingelezen dan er in de string kunnen worden ondergebracht. Meestal declareren we een string met 100 karakters en hopen maar dat er niet meer karakters worden ingevoerd.

We kunnen dit oplossen door on-the-fly dynamisch geheugen te alloceren. Een eenvoudig voorbeeld van dynamische geheugenallocatie is hieronder gegeven:

---

```
1 #include <stdio.h>
2 #include <malloc.h>
3
4 int main(void) {
5
6     char *ptr = malloc(1024); /* Allocate 1024 bytes */
7
8     if (ptr == NULL) {
9         printf("Oops, no memory!\n");
10        exit(-2);
11    }
12
13    printf("Allocated 1024 bytes at address %p\n", ptr);
14
15    free(ptr); /* Free memory */
16
17    return 0;
18 }
```

---

Merk op dat het stuk geheugen dat beschikbaar is gesteld niet is geïnitieerd (bijvoorbeeld met null-bytes).

We hebben nu een stuk geheugen van 1024 bytes. Dat kan op een gegeven moment vol zijn. Er moet dan een stuk geheugen worden gealloceerd dat groter is, bijvoorbeeld 2048 bytes. Dat kan met de functie `realloc()`:

---

```
1     char *ptrnew;
2
3     ptrnew = realloc(ptr, 2048);
```

---

Let erop dat `ptrnew` en `ptr` naar twee verschillende adressen *kunnen* wijzen. Als het eerste geheugengebied niet kan worden uitgebreid dan wordt er een nieuw geheugengebied gealloceerd. De functie `realloc` ↩️ () kopieert dan de 1024 bytes van de eerste `alloc()` naar het nieuwe geheugengebied en geeft de pointer `ptr` vrij d.m.v. `free(ptr)`. Je kan meer vinden over `malloc` en `realloc` op <http://www.cplusplus.com/reference/cstdlib/malloc/>.

Schrijf een functie `char *getoneline(int chunksize)` die een regel tekst van het toetsenbord inleest en in een stuk dynamisch gealloceerd geheugen plaatst. De parameter `chunksize` is het aantal bytes waarmee het geheugen moet worden uitgebreid als een eerder gealloceerd stuk geheugen vol is. Nadat de regel is ingelezen (end-of-line-karakter wordt niet opgeslagen, de null-byte `'\0'` wel) geeft de functie de pointer terug naar het gealloceerde geheugen. Test je functie met een klein programma en varieer de waarde van `chunksize`.

## 8 Bestanden

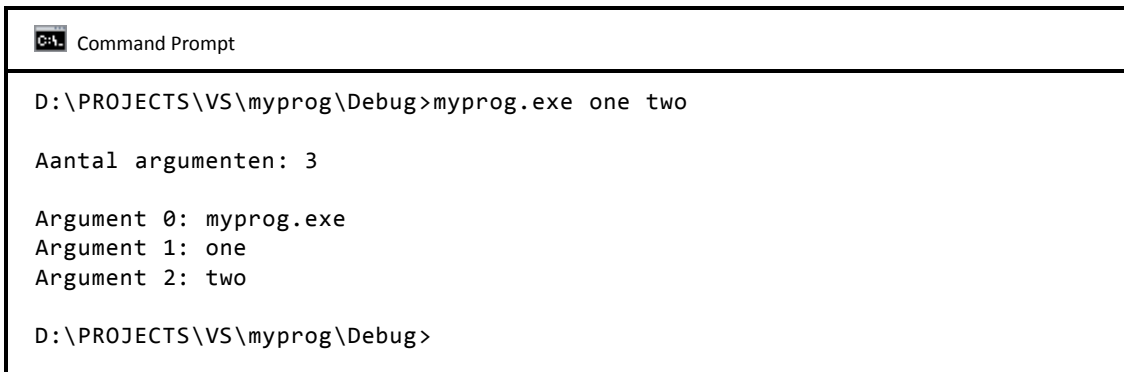
### 8.1 Inleiding

Bij deze opdracht gaan we gebruik maken van argumenten op de command-line en openen, lezen, schrijven en sluiten van bestanden op de disk. Een volledige introductie over deze onderwerpen is te vinden in het boek "De programmeertaal C", hoofdstuk 9. Hieronder wordt slechts het afdrukken van de argumenten op de command-line besproken.

Hieronder zie je een voorbeeld van een programma dat de opgegeven argumenten op het beeldscherm afdruckt. Het programma drukt eerst het aantal opgegeven argumenten af (via de variabele `argc`) en drukt daarna een voor een de argumenten af (via de variabele `argv`). Kopieer de code in een nieuw Visual Studio-project en start het programma vanaf de command-line.

```
1  /* myprog.c */
2  #include <stdio.h>
3  #include <stdlib.h>
4
5  int main(int argc, char *argv[]) {
6
7      int i;
8
9      printf("\nAantal argumenten: %d\n\n", argc);
10     for (i = 0; i < argc; i++) {
11         printf("Argument %d: %s\n", i, argv[i]);
12     }
13     return 0;
14 }
```

Als je dit programma draait vanuit een DOS-box<sup>9</sup>, dan zie je het volgende op het scherm.



```
Command Prompt

D:\PROJECTS\VS\myprog\Debug>myprog.exe one two

Aantal argumenten: 3

Argument 0: myprog.exe
Argument 1: one
Argument 2: two

D:\PROJECTS\VS\myprog\Debug>
```



LEES DE VOETNOOT AAN HET EINDE VAN DIT HOOFDSTUK!.



Let op! Je moet één van de onderstaande opdrachten maken. De practicumdocent bepaalt welke opdracht je moet maken.

<sup>9</sup> Zie [https://en.wikipedia.org/wiki/Win32\\_console](https://en.wikipedia.org/wiki/Win32_console)

## 8.2 More

Schrijf een programma dat de inhoud van een tekstbestand met twintig regels tegelijk op het scherm laat zien. Na een druk op de enter-toets worden de volgende twintig regels getoond. Lever een tekstbestand mee van minimaal 100 regels. Bedenk dat een bestand niet altijd een veelvoud van twintig regels heeft. De naam van het tekstbestand wordt als argument op de command-line\*) meegegeven.

## 8.3 Cat

Schrijf een programma dat de inhoud van een aantal tekstbestanden achter elkaar op het scherm afdrukt. De namen van het tekstbestanden worden als argumenten op de command-line\*) meegegeven. Het aantal bestandsnamen kan variëren, er wordt dus geen vast aantal namen opgegeven. Als geen bestandsnaam wordt opgegeven, moet het toetsenbord als invoer dienen (er kunnen dan geen bestanden geopend worden want er zijn geen namen opgegeven). Als het eerste argument -h is, moet er een helptekst worden afgedrukt en moet het programma stoppen. Er worden dan geen bestanden afgedrukt.

## 8.4 Zinslengte sorteren

Schrijf een programma dat de regels in een tekstbestand inleest en op lengte sorteert. Het resultaat schrijf je naar het scherm en naar een uitvoerbestand. De namen van de tekstbestanden moeten als argumenten op de command-line\*) meegegeven worden. Ga ervan uit dat maximaal 1000 woorden kunnen worden opgeslagen.

## 8.5 Op alfabet sorteren

Schrijf een programma dat de woorden in een tekstbestand op alfabet sorteert. Het resultaat schrijf je naar het scherm en naar een uitvoerbestand. De namen van de tekstbestanden moeten als argumenten op de command-line\*) meegegeven worden.

## 8.6 Regelnummering

Schrijf een programma dat een tekstbestand inleest (bijv. een C-bestand) en hier regelnummers aan toevoegt en naar een uitvoertekstbestand schrijft. Lege regels worden niet genummerd, maar worden wel naar het uitvoerbestand geschreven. De regelnummers komen aan de linkerkant te staan. Gebruik een invoerbestand van minimaal 100 regels tekst en lever dit bestand ook in. De namen van de tekstbestanden moeten als argumenten op de command-line\*) meegegeven worden.

## 8.7 Getallen sorteren

Schrijf een programma dat twee tekstbestanden met ongesorteerde integer getallen samenvoegt tot een derde gesorteerd bestand met integers. De namen van de tekstbestanden wordt als argument op de command-line\*) meegegeven.

## 8.8 Uitgeschreven nummers sorteren

In een tekstbestand staat een willekeurige rij met nummers van nul tot en met twintig. De nummers staan echter niet op volgorde en zijn languit (dus als woorden) geschreven. Sorteert alle woorden (nummers) op numerieke volgorde van hoog naar laag en schrijf het resultaat (aantal keer) naar een nieuw bestand. Het invoerbestand kan er als volgt uitzien:

---

```
1 Twintig
2 Negentien
3 Achttien
4 Zeventien
5 Zestien
6 Vijftien
```

---

Het resulterende bestand kan er dus zo uitzien:

---

```
1 1 Vijftien
2 3 Zestien
3 20 Zeventien
4 5 Achttien
```

---

De namen van de tekstbestanden wordt als argument op de command-line\*) meegegeven.

## 8.9 C keywords

Schrijf een programma dat een tekstbestand met C-programmacode inleest. Het programma moet vervolgens tellen hoe vaak een keyword van de programmeertaal C voorkomt. Vervolgens moet het resultaat weggeschreven worden naar een ander bestand. De namen van de tekstbestanden moeten als argumenten op de command-line\*) meegegeven worden. Het resulterende bestand kan er dus zo uitzien:

---

```
1 2 double
2 12 for
3 5 if
4 8 int
5 6 while
```

---

## 8.10 Commentaar verwijderen

Schrijf een programma dat een tekstbestand met C-programmacode inleest en de programmacode zonder commentaren naar een ander bestand schrijft. Filter zowel blokcommentaren `/*...*/` als lijncommentaren `//...`  uit. De namen van de tekstbestanden moeten als argumenten op de command-line\*) meegegeven worden.

## 8.11 Letters tellen

Schrijf een programma dat een tekstbestand inleest en bepaalt hoe vaak elke letter voorkomt. Lege regels, cijfers en leestekens mogen worden genegeerd. Hoofdletters en bijbehorende kleine letters moeten als identiek beschouwd worden. De resultaten moeten naast absolute aantallen ook als percentages van het totaal aantal letters in een tekstbestand worden weggeschreven. Maak je code efficiënt door gebruik te maken van arrays. De namen van de tekstbestanden moeten als argumenten op de command-line\*) meegegeven worden.

## 8.12 Woorden tellen, en meer

Schrijf een programma dat een tekstbestand inleest en bepaalt hoeveel woorden, regels en karakters het bestand telt. Het resultaat moet een tweede tekstbestand worden ingelezen. De namen van de tekstbestanden moeten als argumenten op de command-line\*) meegegeven worden.

## 8.13 Omkeren

Schrijf een programma dat een tekstbestand per regel inleest en de regels achterstevoren in een tweede tekstbestand plaatst. De namen van de tekstbestanden moeten als argumenten op de command-line\*) meegegeven worden.

## 8.14 Verwijderen

Schrijf een programma dat een tekstbestand inleest en alle spaties aan het begin en het einde van een regel verwijdert. Verder moeten alle meervoudige spaties tussen de woorden vervangen worden door één spatie. De uitvoer moet naar een tweede bestand geschreven worden. De namen van de tekstbestanden moeten als argumenten op de command-line\*) meegegeven worden.

## 8.15 XeT

Schrijf een programma dat een tekstbestand inleest. In het tekstbestand zitten speciale commando's verborgen. Deze zijn:

- Als het commando `\spaces<getal>` wordt gevonden, dan moet dit commando vervangen worden door een aantal spaties dat is opgegeven met het getal `<getal>`. De tekens `<` en `>` zijn geen onderdeel van het commando. Merk op dat tussen `\spaces` en het getal geen spaties staan. Na het getal kan een spatie staan. Als er geen getal wordt gevonden, moet een foutmelding op het scherm volgen.
- Als het commando `\lines<getal>` wordt gevonden, dan moet dit commando vervangen worden door een aantal lege regels dat is opgegeven met het getal `<getal>`. De tekens `<` en `>` zijn geen onderdeel van het commando. Merk op dat tussen `\lines` en het getal geen spaties staan. Na het getal kan een spatie staan. Als er geen getal wordt gevonden, moet een foutmelding op het scherm volgen.
- Als het commando `\nobreak` wordt gevonden, dan moet het end-of-line-karakter aan het einde van de huidige ingelezen regel verwijderd worden. Het end-of-line-karakter wordt dus niet afgedrukt. Merk op dat na `\nobreak` altijd een spatie of een end-of-line-karakter moet volgen anders is het commando `\nobreak` niet te detecteren.

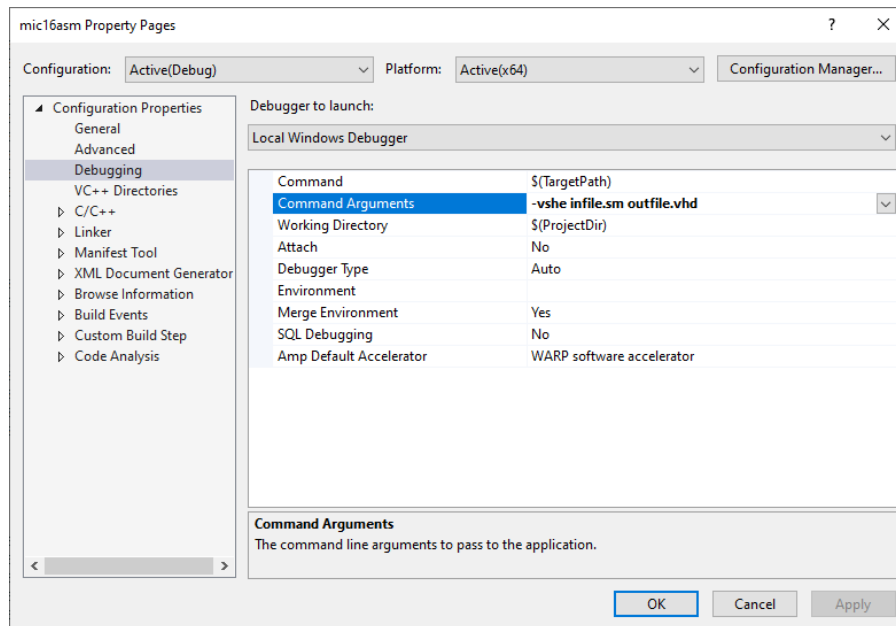
De aangepaste tekst moet naar een tekstbestand geschreven worden. Eventuele foutmeldingen moeten naar het scherm worden geschreven. Ga ervan uit dat een ingelezen regel uit niet meer dan 100 karakters bestaat. De namen van de tekstbestanden wordt als argument op de command-line\*) meegegeven.

## 8.16 \*) Argumenten op de command-line

Met argumenten op de command-line wordt het volgende bedoeld:

- Open een command-box (Windows: Windowstoets-R en type `cmd.exe`, OS-X: start terminal; Linux: start terminal).
- Navigeer naar de map waar de *executable* (= uitvoerbaar programma) staat. Bij Visual Studio iets van `C:\Users\...\source\repos\...\Debug`. Bij Linux/Mac: effe uitzoeken.
- Start de executable met `naamvanexecutable.exe filename1.txt filename2.txt <enter>` (of zoveel argumenten als noodzakelijk).
- Het programma start nu met de twee argumenten.

Het is ook mogelijk om in Visual Studio de command line parameters in te stellen. Open hiervoor je project. Navigeer dan naar **Project** → **Properties**. Selecteer links in het geopende dialoogvenster het onderdeel **Debugging**. Vul de gegevens in bij **Command arguments**. Zie figuur 2.



Figuur 2: Instellen command line parameters.

Het is dus **niet** de bedoeling dat na het starten van het programma de gebruiker om twee bestandsnamen wordt gevraagd of dat de bestandsnamen hard gecodeerd zijn in het programma. Verder moet getest worden of de bestanden wel geopend kunnen worden. Zo niet, dan moet dat aan de gebruiker worden medegedeeld. Als een gebruiker je programma start met teveel of te weinig argumenten, dan moet je dat aan de gebruiker mededelen. Ga er niet vanuit dat de gebruiker de inhoud van de bestanden zo formatteert dat het direct in je programma werkt. Je moet dus de ingelezen data wel controleren of het voldoet aan het formaat dat je kan verwerken (denk bijvoorbeeld aan letters i.p.v. getallen). Maak je programma flexibel. Dus niet voorschrijven dat je precies 10 getallen moet invoeren (maar om het eenvoudig te houden: niet meer dan 1000) of dat een regel precies 20 karakters moet hebben. Bedenk ook wat er moet gebeuren als een invoerbestand leeg is.

## A Installatie Visual Studio Community

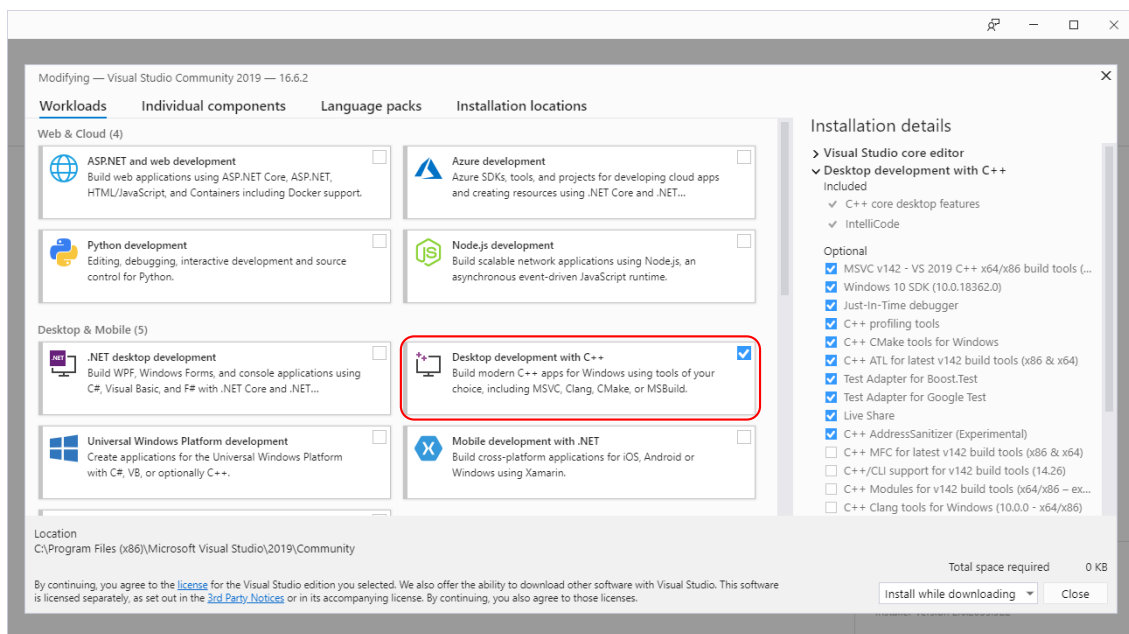
We behandelen in deze bijlage het opzetten van een project in Visual Studio 2019. Visual Studio 2019 kan gedownload worden van de website van Microsoft. De precieze installatie van de software verschilt van computer tot computer, afhankelijk van de al eerder geïnstalleerde software, met name de *runtime libraries*.

Download Visual Studio Community via de link:

<https://visualstudio.microsoft.com/thank-you-downloading-visual-studio/?sku=Community&channel=Release&version=VS2022&source=VSLandingPage&cid=2030&passive=false>

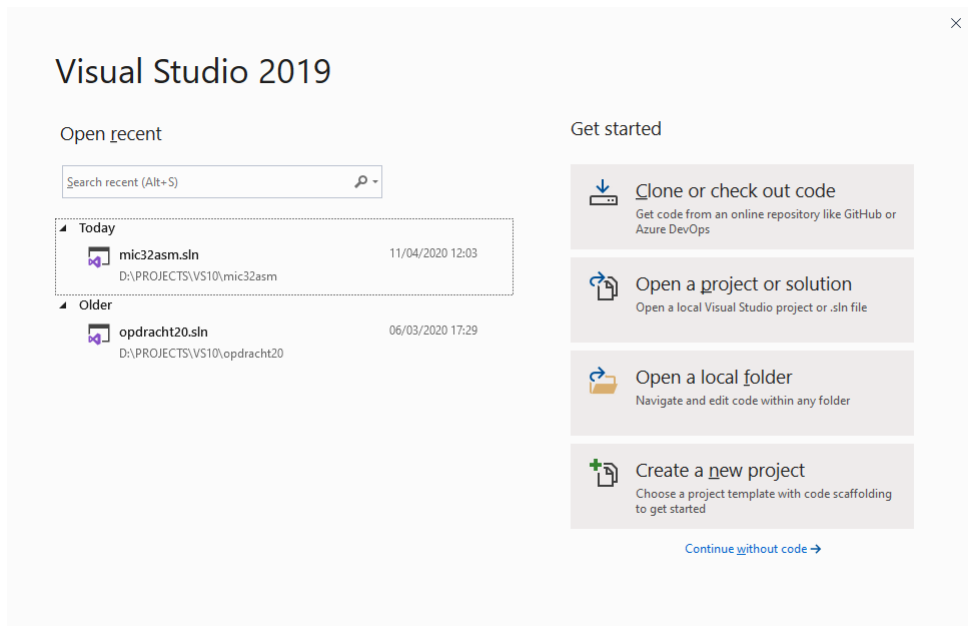
Er wordt nu een kleine installer gedownload. Start daarna de installer. Visual Studio is een groot programma, dus het kost nogal wat tijd om Visual Studio te installeren.

Visual Studio ondersteunt het ontwikkelen van applicaties met een veelzijdigheid aan mogelijkheden. Wij gebruiken de C/C++-compiler voor Desktop Development. Om te kunnen ontwikkelen moeten we een *workload* installeren. Selecteer de workload zoals te zien is in figuur 3. Selecteer de opties zoals is aangegeven aan de rechterkant van de figuur. Klik daarna op **Install while downloading**.



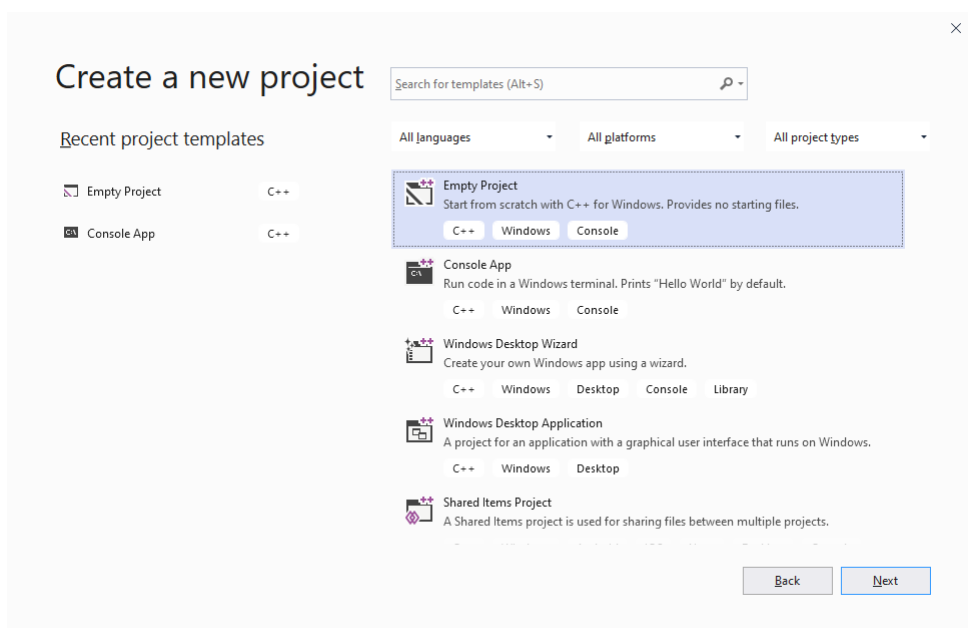
Figuur 3: Selecteren van de C++-workload.

Start Visual Studio door op het icoon te klikken. Visual Studio opent een beginscherm waarin een nieuw project kan worden aangemaakt. Dit is te zien in figuur 4. Klik op het kader **Create a new project**.



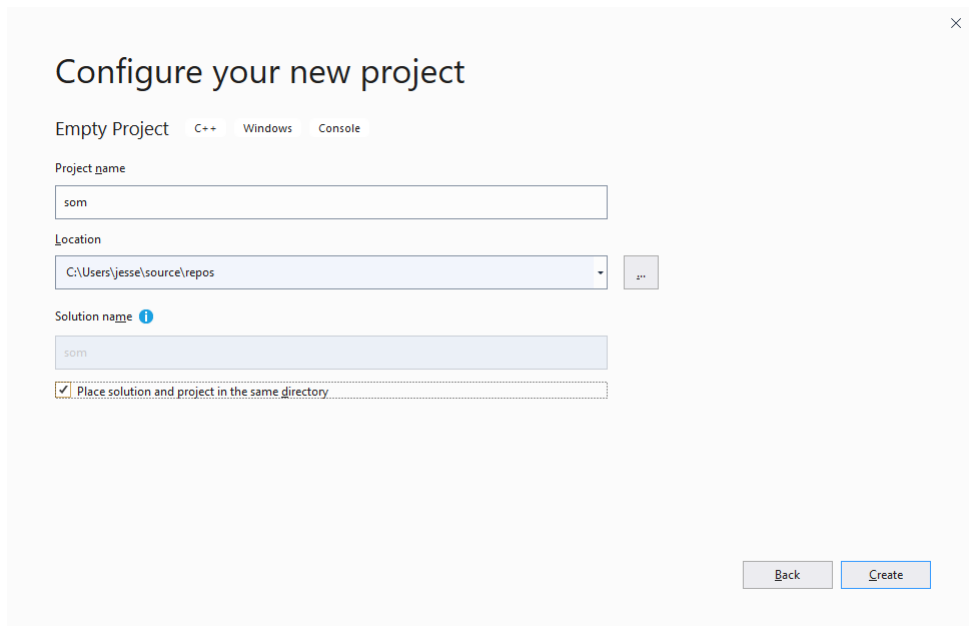
Figuur 4: Aanmaken van een nieuw project.

Er wordt een nieuw scherm geopend, zie figuur 5. Klik daarin op het kader **Empty Project**. **Klik niet op Console App.**



Figuur 5: Aanmaken van een leeg project.

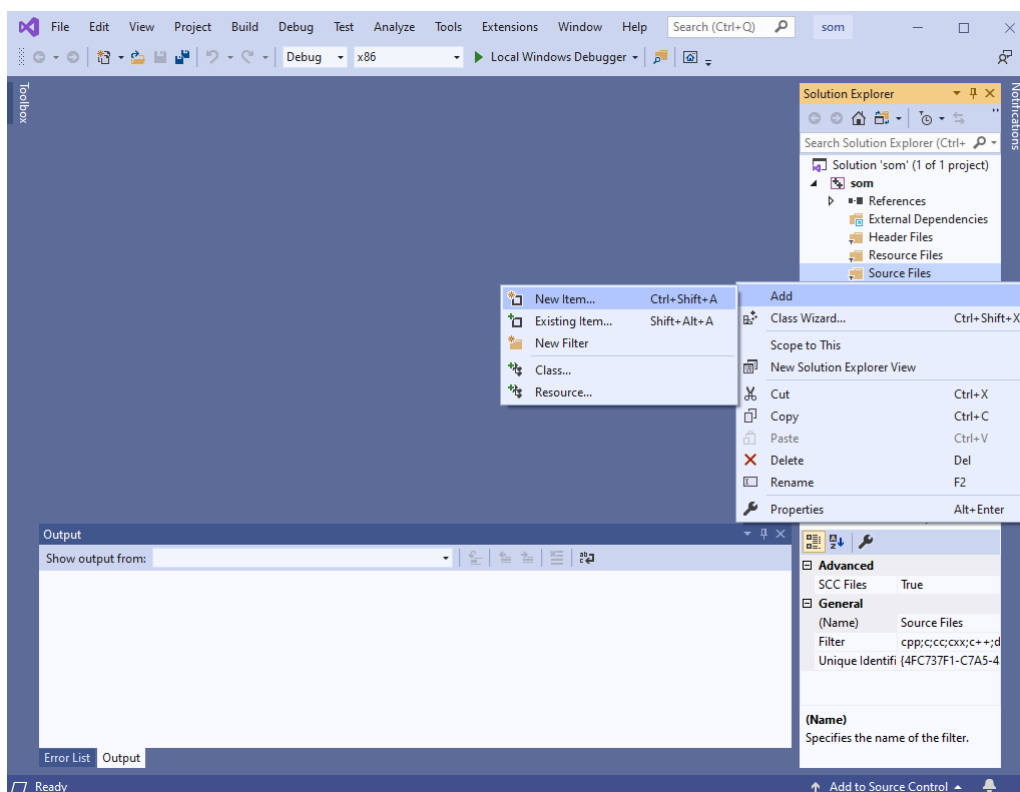
Daarna moeten wat gegevens worden ingevuld. Vul de projectnaam in en de map waarin het project terecht moet komen. Vink de checkbox onderaan aan en klik op de knop Create. Zie figuur 6.



Figuur 6: Gegevens van het project invoeren.

Visual Studio komt nu met het hoofdscherm waarin een aantal vensters (Engels: pane) te zien zijn. Er is nog geen C-bestand aangemaakt, dat moeten we zelf doen. In de *Solution Explorer* aan de rechterkant is een map *Source Files* te zien. Ga met de muis-pointer daar op staan en klik op de **rechter** muisknop.

Selecteer daarna de optie *Add* en daarna *New item...* . Zie figuur 7.



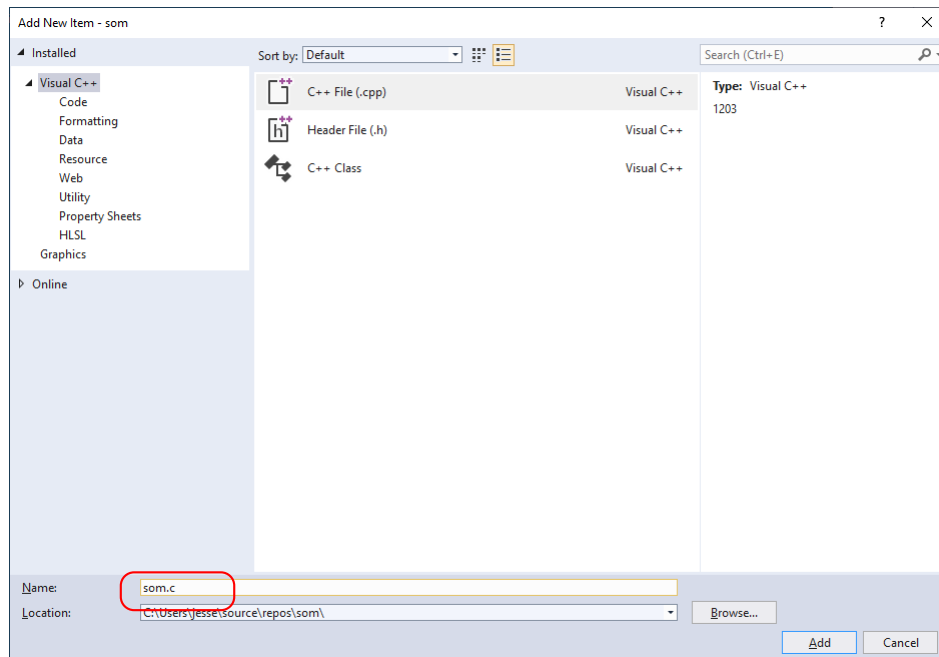
Figuur 7: Een nieuw bestand aanmaken.

In het volgende scherm moet een bestandstype en een naam worden opgegeven. Klik op C++ File (.cpp). Vul onderin bij Name de naam van het bestand in **en zorg ervoor dat de naam eindigt met .c**, anders wordt een C++-bestand aangemaakt. Klik daarna op de knop Add. Zie figuur 8.

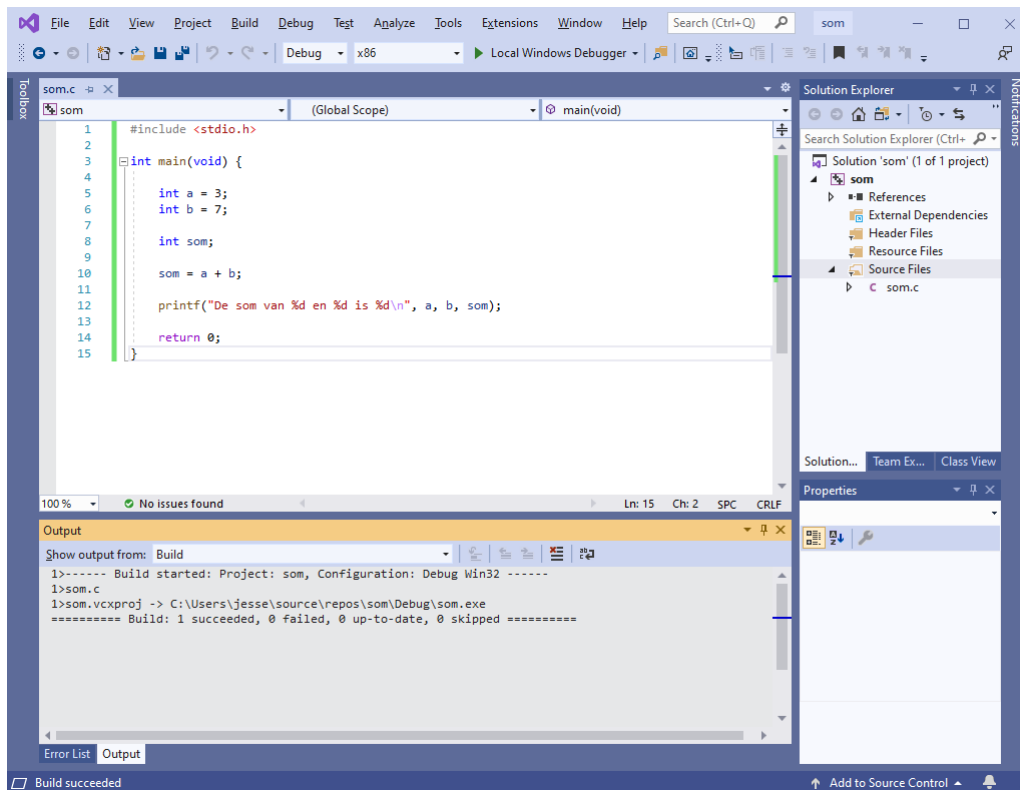
Voer het programma in zoals te zien is in figuur 9. Klik daarna op de knop Local Windows Debugger. Het programma wordt nu gecompileerd en als er geen fouten zijn gevonden, wordt het programma uitgevoerd. Herstel eventuele fouten die door de compiler gevonden worden en herstart de compilatie.

Het programma drukt de regel De som van 3 en 7 is 10 af. Dit wordt gedaan in een zogenoemde *console*. Dit is te zien in figuur 10.

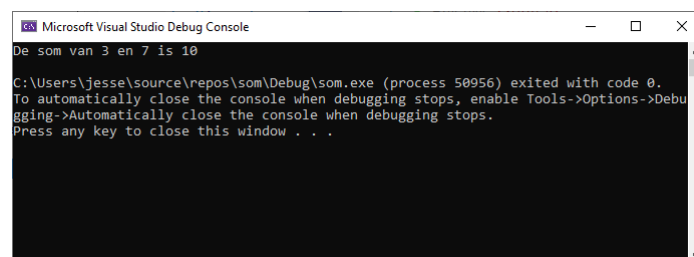
De tutorial is hiermee ten einde.



Figuur 8: Gegevens van het C-bestand invullen.



Figuur 9: Compileren en starten van de executable.



Figuur 10: Uitvoer van het programma in een console.

## B Installatie Code::Blocks

In deze bijlage zijn instructies te vinden voor het installeren en gebruiken van Code::Blocks 20.03. De instructies zijn als volgt:

- Download Code::Blocks van <https://sourceforge.net/projects/codeblocks/files/Binaries/25.03/Windows/codeblocks-25.03mingw-setup.exe/download> (let erop dat in de naam de term mingw staat)
- Installeer vervolgens Code::Blocks door de gedownloade executable te runnen. (Bij installatie continu op next drukken)
- Bij het opstarten van Code::Blocks wordt er gevraagd naar de compiler die gebruikt zal gaan worden. Als het goed is, is er al één geselecteerd en hoeft je alleen op OK te drukken.
- Nu kun je een project aanmaken en code gaan compileren en runnen:
  - a. File->New->Project
  - b. Selecteer Console application + GO
  - c. Next
  - d. Selecteer C (!!!NIET!!! C++)
  - e. Project Title = helloworld
  - f. Folder to create project in: Selecteer een map die je later nog terug kan vinden...
  - g. Next
  - h. Finish (vinkjes aan, GNU GCC compiler geselecteerd)
  - i. Links in de projectomgeving is nu de file main.c te vinden met code:

---

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 int main(void)
5 {
6     printf("Hello world\n");
7     return 0;
8 }
```

---

- j. Klik op Build (Ctrl+F9) (het tandwiel symbool bovenaan)
- k. Klik op Run (Ctrl+F10) (het groene play symbool bovenaan)